



**INNOPOLIS
UNIVERSITY**

Введение в компьютерное зрение

Илья Афанасьев, к.т.н., ЛИРС

Лекция 6 · Частотная область и пирамиды

Dr. Ilya Afanasyev, i.afanasyev@innopolis.ru

«Introduction to Computer Vision» course, Innopolis University (IU), Russia, Fall 2026

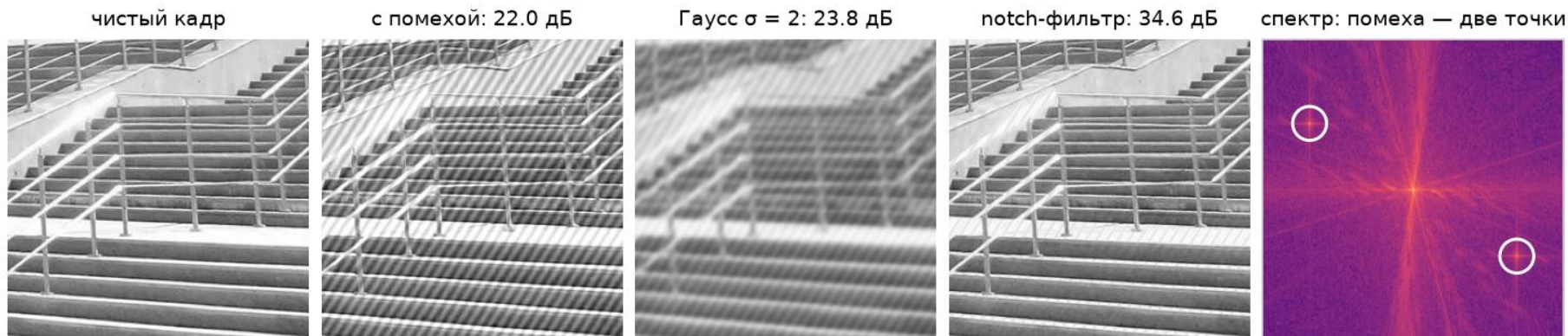
Блок I Изображение и его источник	Блок II Операции над изображением	Блок III Признаки	Блок IV Гомография и геометрия	Блок V Видео и движение
L1 Введение. Изображение и видео как данные	L4 Геометрические преобразования	L7 Границы, линии и робастное оценивание	L10 Гомография: оценивание и выравнивание	L13 Видео: движение и оптический поток
L2 Формирование изображения: оптика и сенсор	L5 Пространственная фильтрация	L8 Точки интереса: детекторы	L11 Модель камеры и калибровка	L14 Трекинг и распознавание объектов
L3 Цвет, гистограммы и точечные операции	L6 Частотная область и пирамиды	L9 Дескрипторы и сопоставление	L12 Стерео и эппиполярная геометрия	L15 Синтез: от двух видов к 3D. Итоги

Сегодня — L6: Частотная область и пирамиды

МОТИВАЦИЯ

Дано: кадр с периодической помехой, 22.0 дБ. **Найти:** кадр без полос и с целыми ступенями

- **Пространственные фильтры не справляются:** плохо подавляют полосы, либо сильно размывают важные детали изображения (например, ступени лестницы)
- **Робототехника:** наводка от моторов в видеотракте, мерцание ламп · **МОИИ:** растр сканов, муар на фото экрана



Один кроп лестницы: чистый кадр, помеха, лучший Гаусс, режекторный фильтр — и спектр, в котором помеха видна как две точки

Пространственные фильтры (размытие по Гауссу, медианный, билатеральный) неэффективны против периодических помех (полос) на изображении, тогда как фильтрация в частотной области (удаление пиков в спектре) позволяет эффективно убрать шум, сохранив четкость деталей.

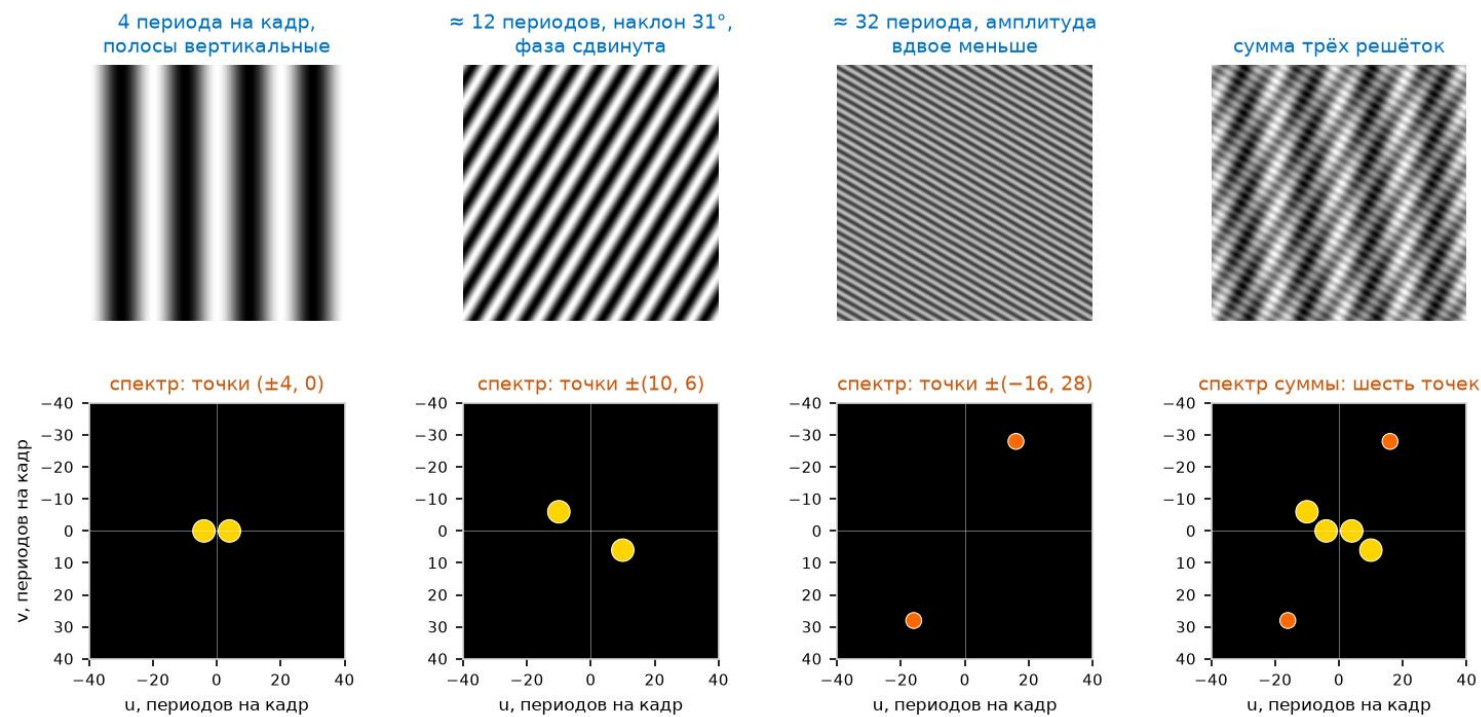
© фото — здание Университета Иннополис; собственная иллюстрация

1

Частотная область

- Кадр как сумма волн · амплитуда и фаза · теорема о свёртке · фильтры в частотах · теорема отсчётов и алиасинг

Кадр как сумма решёток



Принцип суперпозиции:
Сложение трёх разных решёток даёт сложное текстурированное изображение, а в его спектре появляются ровно **шесть точек** (три пары).

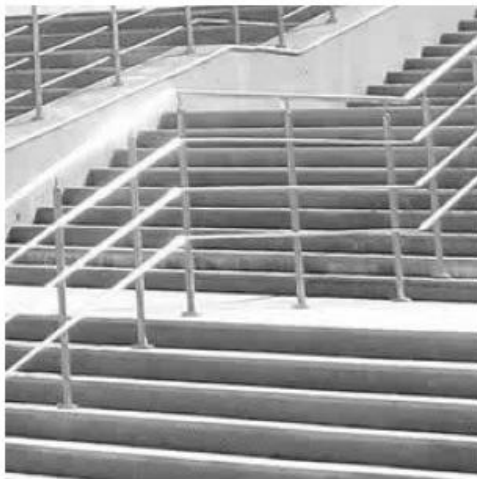
Решётка — синусоида на плоскости: у неё есть частота, направление, амплитуда и фаза. В спектре она — пара точек: чем дальше от центра, тем мельче полосы; направление на точку — поперёк полос; яркость — амплитуда. Сумма трёх решёток даёт шесть точек: преобразование Фурье раскладывает на такие решётки любой кадр

Любое изображение (кадр) можно представить как сумму элементарных синусоидальных решёток, а преобразование Фурье раскладывает кадр на эти составляющие, где каждая решётка отображается в спектре в виде пары симметричных точек

© собственная иллюстрация

Где это работает

JPEG: DCT блоков 8×8 ,
10 % коэффициентов — 38.1 дБ



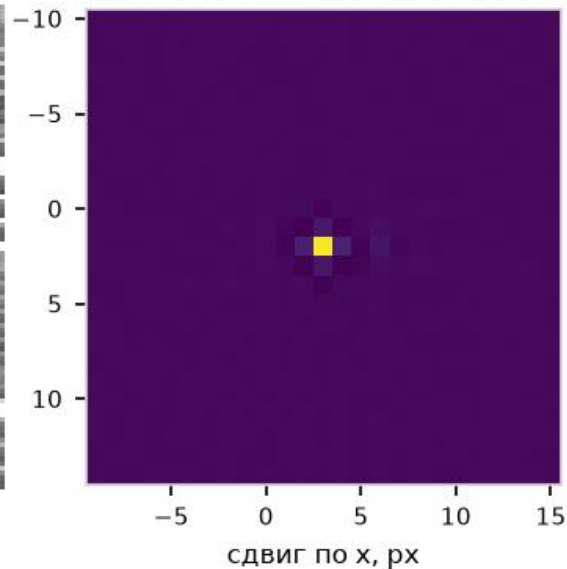
Сжатие изображений (JPEG):
хранения всего 10%
важнейших частотных
коэффициентов достаточно
для восстановления кадра с
отличным качеством (38.1 дБ)

уменьшение $\times \frac{1}{4}$: без фильтра
21.9 дБ, с фильтром (внизу) 37.2



Ресайз: уменьшение картинки
 $4\times$ без предварительной
частотной фильтрации делает
муар (ложные узоры, 21.9 дБ).
Применение нч-фильтра перед
уменьшением убирает муар

совмещение кадров: пик фазовой
корреляции — сдвиг (3.0, 2.0) px



Выравнивание кадров (Фазовая
корреляция): позволяет по пику
в частотной области находить
взаимный сдвиг между
кадрами (например, на
несколько пикселей).

склейка панорамы по полосам:
шва нет, контуры не дwoятся



Бесшовная склейка панорам:
Сшивание изображений «по полосам
частот» (мультимасштабное
смешивание) позволяет объединять
кадры так, что стыковочный шов
полностью исчезает, а контуры
объектов на границах не дwoятся

Четыре продукта одной идеи: JPEG хранит DCT блоков 8×8 — десятой части коэффициентов хватает на 38.1 дБ; уменьшение $\times \frac{1}{4}$ без фильтра даёт ложные узоры и 21.9 дБ, с фильтром — 37.2; фазовая корреляция находит сдвиг между кадрами панорамы (3.0, 2.0) px; сама панорама склеивается по полосам частот — без шва и без двоения

© фото — здание Университета Иннополис; собственная иллюстрация

Формально

- **Дано:** кадр $I(x, y)$ размером $W \times H$. **Найти:** Спектр $F(u, v) = \sum I(x, y) \cdot \exp(-2\pi i (ux/W + vy/H))$ — комплексное число для каждой пары частот
- **Что это значит физически:** $|F(u, v)|$ — сколько в кадре решётки с u периодами по ширине и v по высоте; $\arg F$ — где лежат её гребни
- **Единицы:** периодов на кадр или циклов на пиксель; предел — **0.5 цикла на пиксель**, период $2 p_x$ — частота Найквиста
- **Критерий:** преобразование обратимо — $\text{ifft2}(\text{fft2}(I))$ возвращает кадр с ошибкой $5 \cdot 10^{-13}$

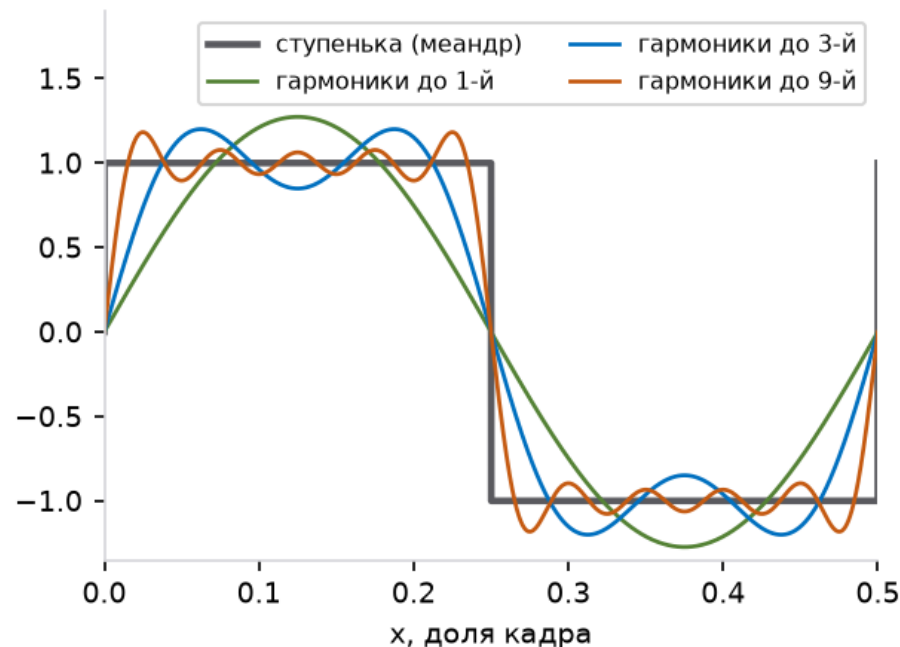
Спектр — тот же кадр в другом базисе: 2.5 млн пикселей $\leftrightarrow$ 2.5 млн коэффициентов

$F(0, 0)$ — сумма всех пикселей, $3.3 \cdot 10^8$: «нулевая частота», или DC

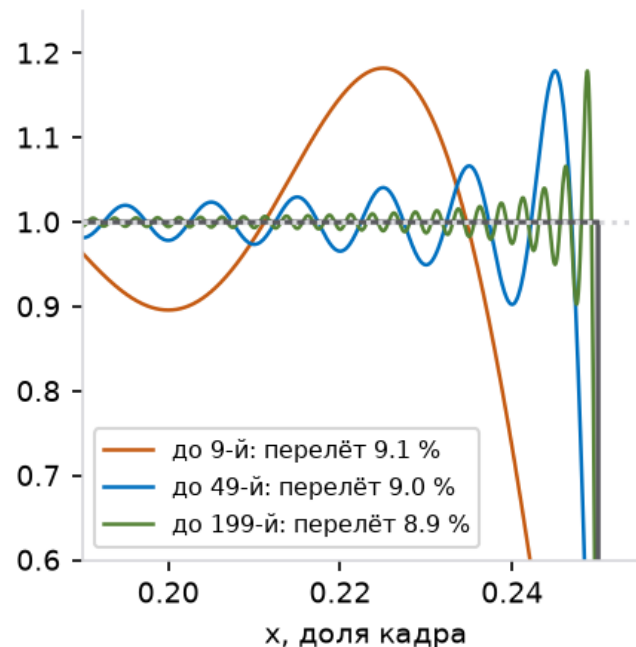
Дискретное преобразование Фурье (ДПФ) изображения переводит кадр из пространственного базиса в частотный без потери информации.

Резкий край из синусоид

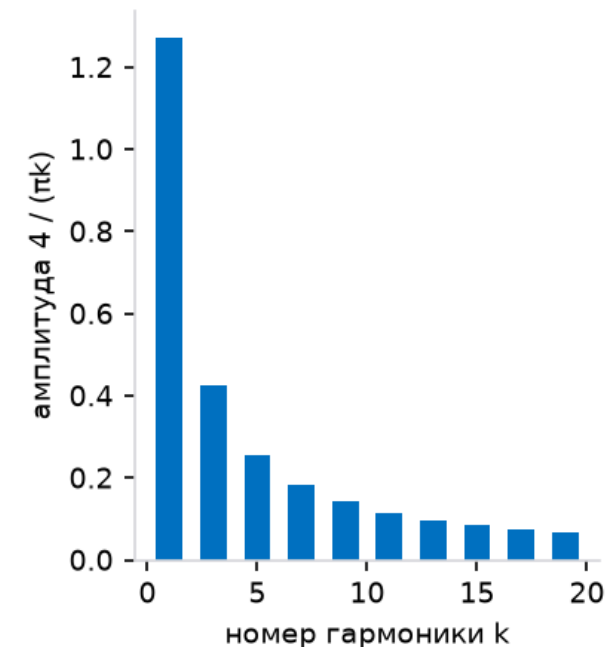
резкий край собирается из синусоид: 1, 1/3, 1/5, ... по амплитуде



у края перелёт $\approx 9\%$ скачка — не исчезает, только сужается



спектр ступеньки: спадает как $1/k$



Ступенька — сумма нечётных гармоник с амплитудами 1, 1/3, 1/5...: чем резче край, тем больше высоких частот в нём. Оборвали ряд — у края остаётся перелёт около 9 % скачка: на 9-й гармонике 9.1 %, на 199-й — 8.9 %. Перелёт не уменьшается, только сужается: это явление Гиббса, оно вернётся кольцами вокруг краёв

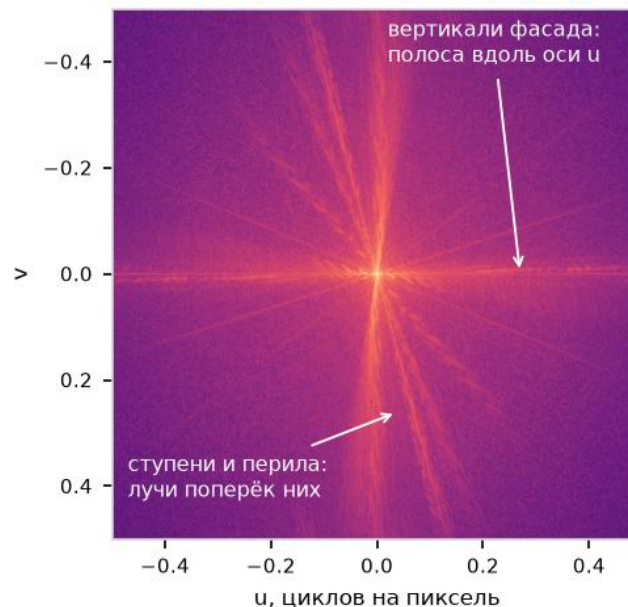
Феномен Гиббса: при разложении резкого перепада яркости (ступеньки/меандра) в ряд Фурье ограничение количества частот приводит к появлению осцилляций и выброса (перелёта) величиной около 9% у края, который не исчезает, а лишь сужается с добавлением новых гармоник

© собственная иллюстрация

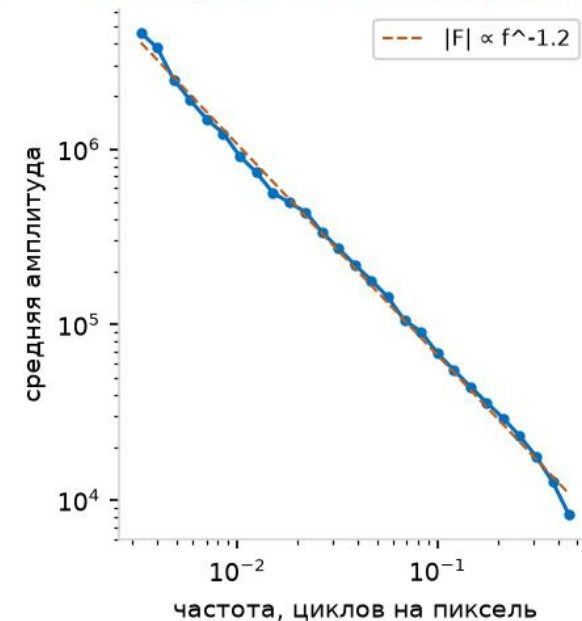
кадр 1920 × 1279



лог-амплитуда $\log(1 + |F|)$: центр — нулевая частота,
край — 0.5 цикла на пиксель (период 2 px)



амплитуда падает с частотой:
80 % энергии — в 0.03 % коэффициентов



Лог-амплитуда после `fftshift`: в центре нулевая частота, к краю — всё более мелкие детали. Амплитуда падает как $f^{-1.2}$: 80 % энергии лежит в 0.03 % коэффициентов у центра. Лучи — следы прямых линий кадра, каждый луч перпендикулярен своим линиям: вертикали фасада дают полосу вдоль оси u , ступени и перила — лучи поперёк них

Двумерный частотный спектр естественного изображения устроен предсказуемо: подавляющая часть его энергии сосредоточена в низких частотах (в центре), а любые контрастные линии и контуры на фотографии оставляют в спектре четкие лучи, направленные строго перпендикулярно этим линиям

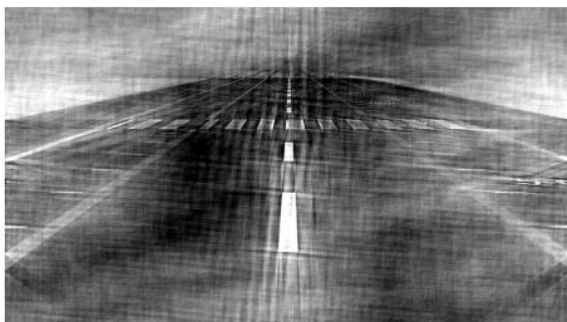
© фото — здание Университета Иннополис; собственная иллюстрация

Фаза важнее амплитуды

кадр A: здание



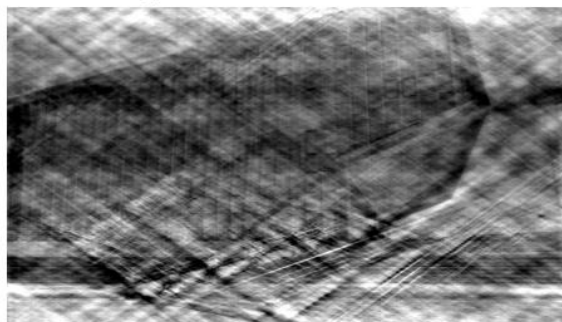
амплитуда A + фаза B: видна дорога
корреляция с B 0.55, с A 0.32



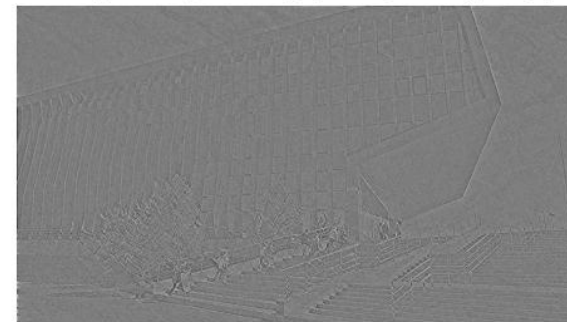
кадр B: дорога с камеры робота



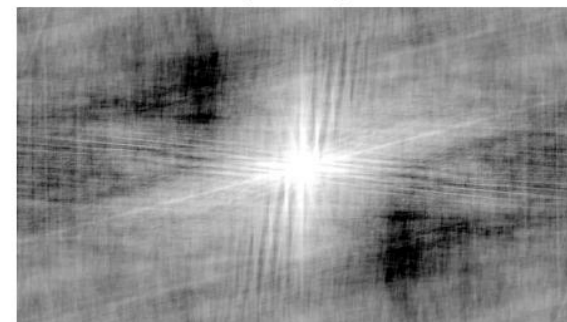
амплитуда B + фаза A: видно здание
корреляция с A 0.55, с B 0.13



только фаза A (амплитуда = 1): контуры здания



только амплитуда A (фаза = 0): пятно



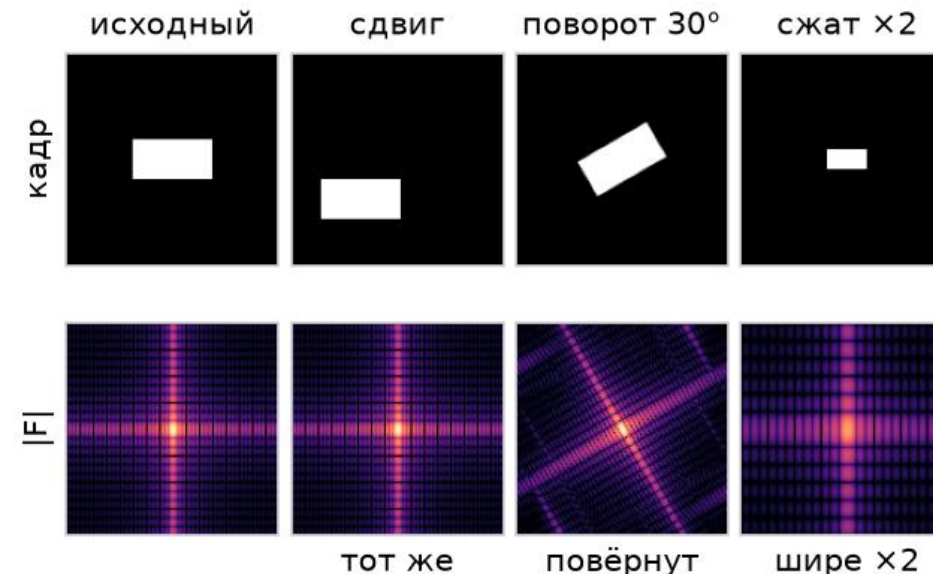
Поменяли местами амплитуду и фазу двух кадров: узнаётся тот кадр, чью фазу взяли, — корреляция с донором фазы 0.55, с донором амплитуды 0.13–0.32. Фаза хранит, где лежат края; амплитуда — только сколько каких частот в кадре. Одна фаза даёт контуры здания, одна амплитуда — пятно

В частотном спектре изображения фазовая информация важнее амплитудной для восприятия структуры кадра, поскольку фаза кодирует пространственное положение контуров и геометрию объектов, тогда как амплитуда отвечает лишь за общую интенсивность частот

© фото — здание Университета Иннополис; кадр с камеры робота — синтетика L4; собственная иллюстрация

Четыре свойства спектра

- **Линейность:** спектр суммы — сумма спектров; помеха, прибавленная к кадру, лежит в спектре отдельно от него
- **Сдвиг меняет только фазу:** амплитуда сдвинутого кадра та же до $7 \cdot 10^{-17}$; на этом стоит фазовая корреляция — `cv2.phaseCorrelate` нашёл сдвиг (3, 2) px
- **Поворот кадра — поворот спектра;** сжали объект вдвое — спектр стал вдвое шире: мелкое в кадре лежит далеко от центра
- **Энергия сохраняется (Парсеваль):** $\sum I^2 = \sum |F|^2 / N$ — $5.8745 \cdot 10^{10}$ с обеих сторон



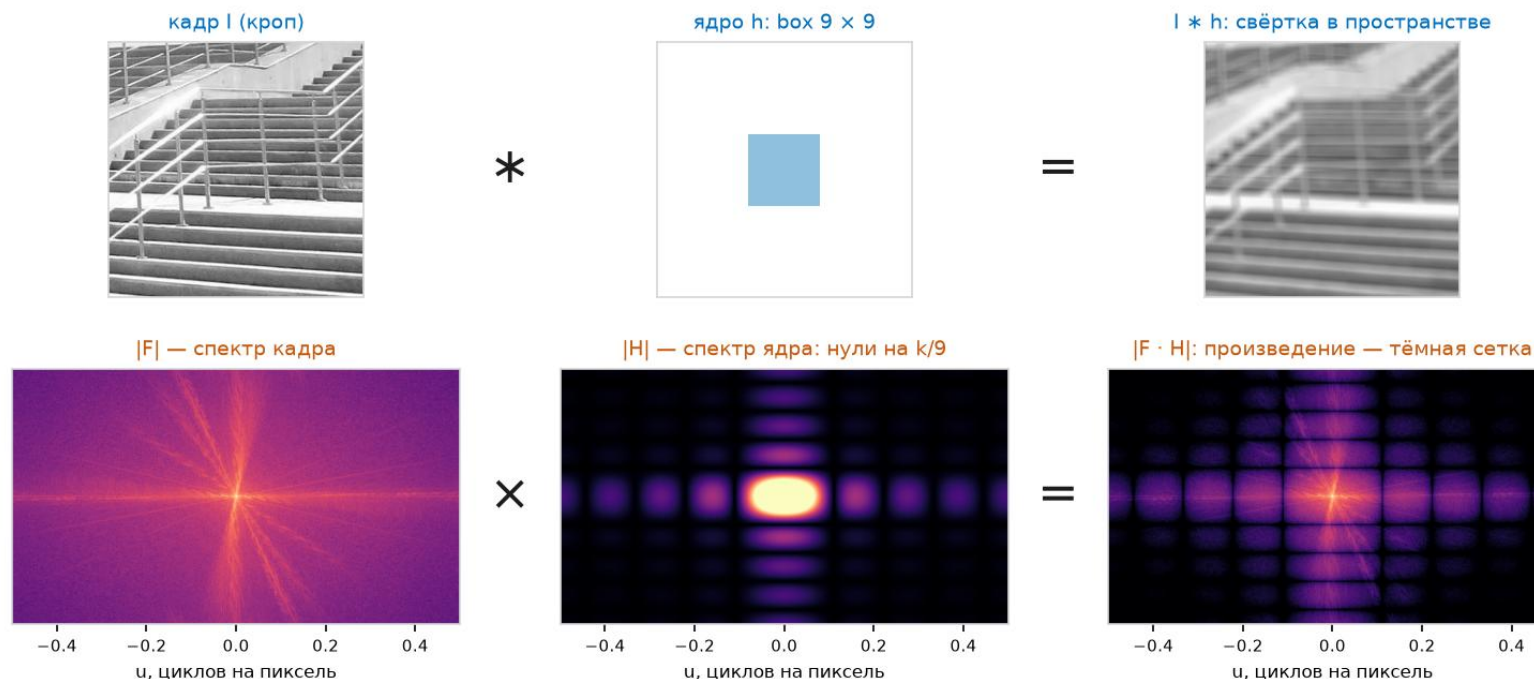
Из линейности следует notch-фильтр, из сдвига — совмещение кадров, из масштаба — связь размера детали с частотой

Поворот и масштабирование: Геометрические преобразования кадра зеркально отражаются в его спектре:

- Поворот кадра на угол α приводит к аналогичному повороту спектра на тот же угол.
- Сжатие объекта в пространственной области (в данном примере вдвое) делает его спектр вдвое шире (мелкие детали на изображении уходят в область высоких частот, расположенных далеко от центра).

© собственная иллюстрация

Теорема о свёртке

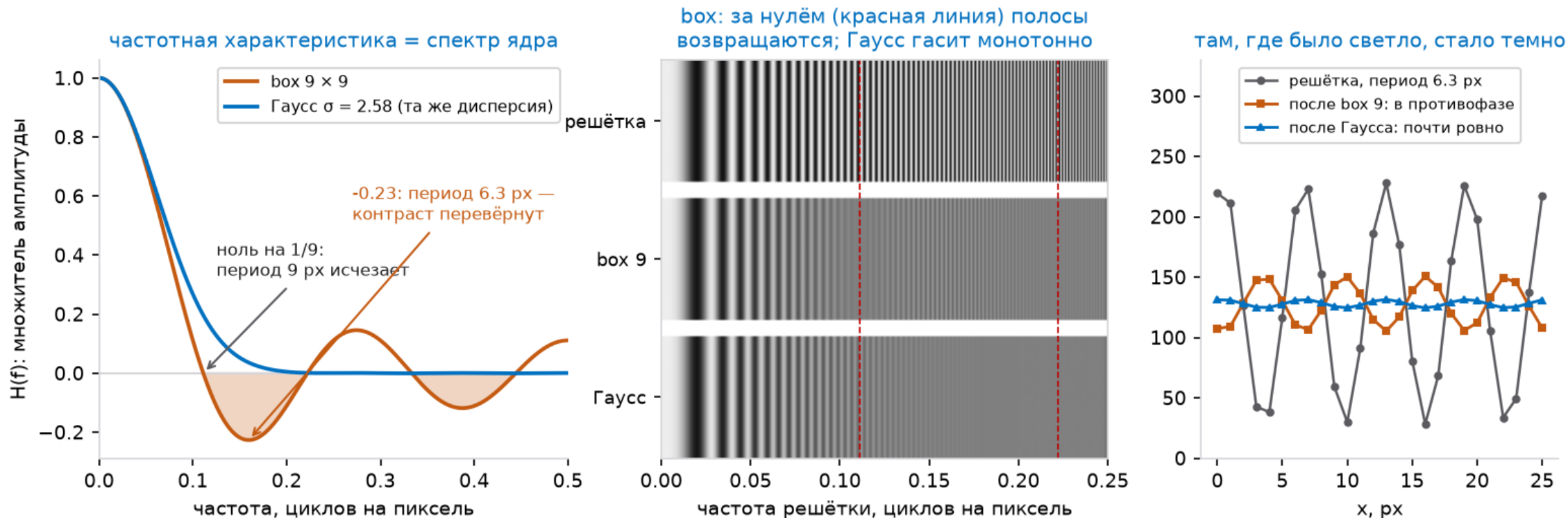


Свёртка в пространстве — умножение спектров: $I * h \leftrightarrow F \cdot H$. Спектр box 9×9 обращается в ноль на частотах $k/9$, и в спектре результата на этих местах тёмная сетка. Проверка числом: `filter2D` и `ifft2(F · H)` совпадают до $5 \cdot 10^{-13}$ внутри кадра; у края расходятся на 112 кодов — спектр считает кадр периодическим

Теорема о свёртке: операция пространственной фильтрации (свёртки) изображения с ядром математически эквивалентна попиксельному умножению их частотных спектров в Фурье-области

© фото — здание Университета Иннополис; собственная иллюстрация

Почему box «звенит»

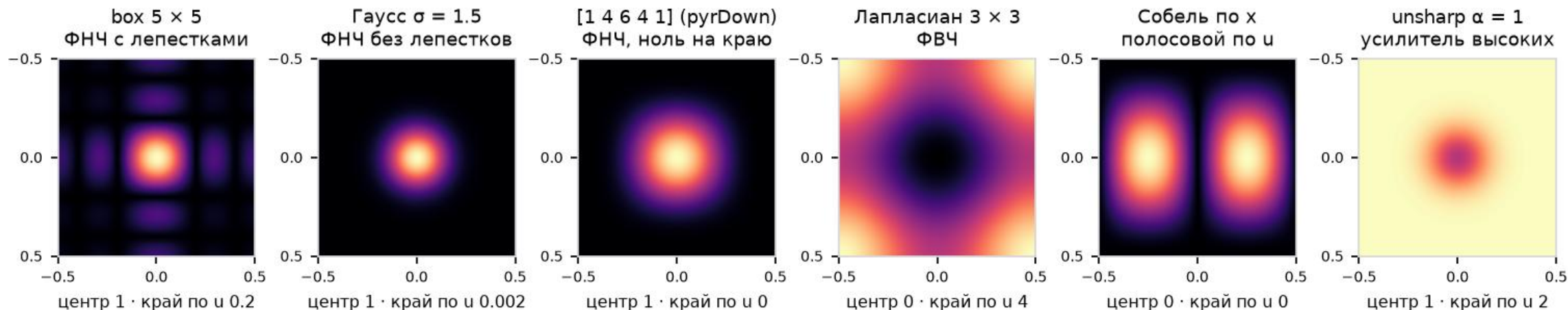


Частотная характеристика — спектр ядра. У box 9×9 она проходит через ноль на частоте $1/9$ и уходит в минус: решётка с периодом 6.3 px выходит с контрастом -0.23 — светлое стало тёмным. Дальше лепесток $+0.15$, на частоте Найквиста остаётся 0.11. Гаусс с той же дисперсией гасит монотонно: на периоде 6.3 px — 0.04

Эффекта «звона» (артефактов размытия и инверсии контраста) прямоугольного фильтра (box filter) через его частотную характеристику, которая колеблется и уходит в отрицательные значения, в отличие от фильтра Гаусса, который подавляет высокие частоты монотонно и без искажений фазы

© собственная иллюстрация

Пространственные ядра в частотной области

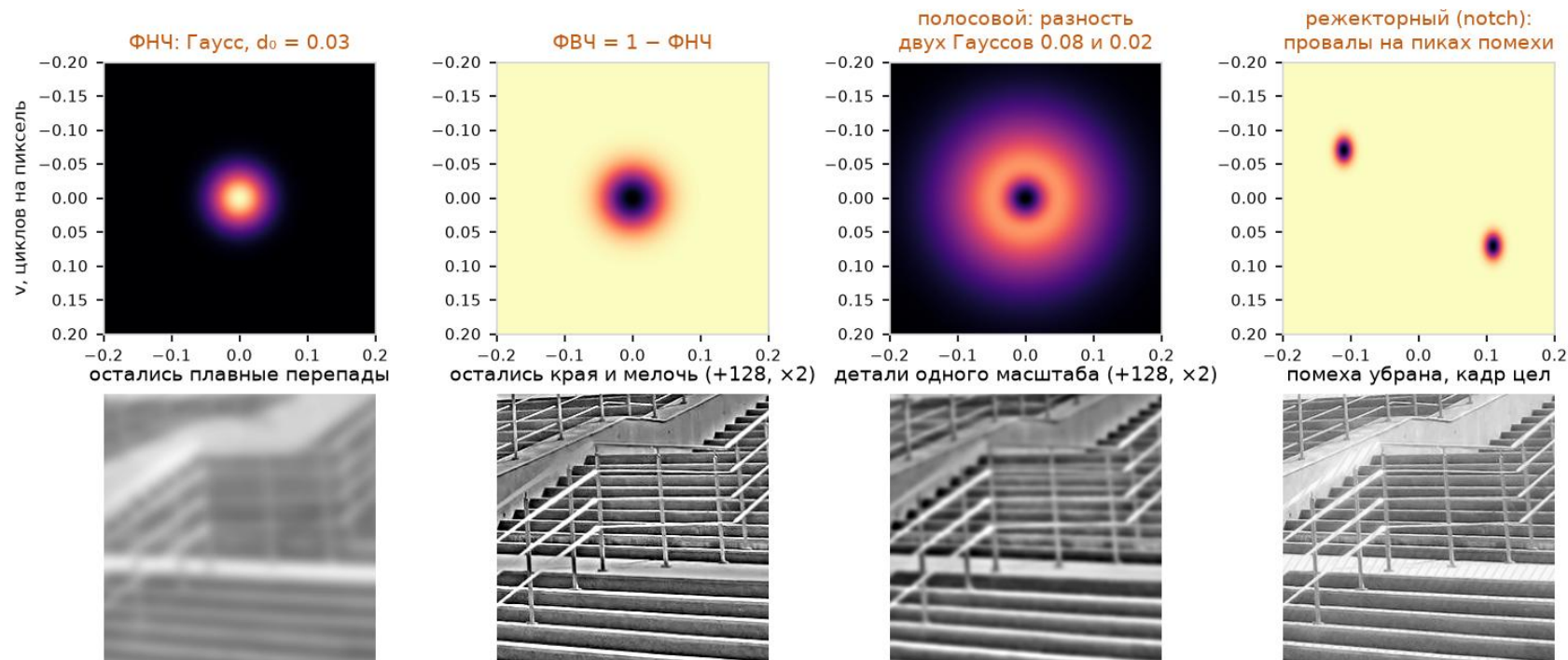


Спектр ядра сразу говорит, что оно делает. Box и Гаусс — фильтры низких частот, у box видны лепестки. Ядро pyrDown гасит частоту 0.5 до нуля. Лапласиан — фильтр высоких: ноль в центре, 4–8 на краю. Собель по x — производная по u и сглаживание по v. Unsharp усиливает высокие вдвое: белый шум после него вырос в 1.94 раза — ровно как предсказывает спектр

Визуальный анализ спектра любого пространственного ядра позволяет мгновенно определить его точную функциональную роль (сглаживание, выделение контуров или усиление резкости), наглядно демонстрируя связь между пиксельными масками и их частотными характеристиками

© собственная иллюстрация

Фильтры, заданные в частотах



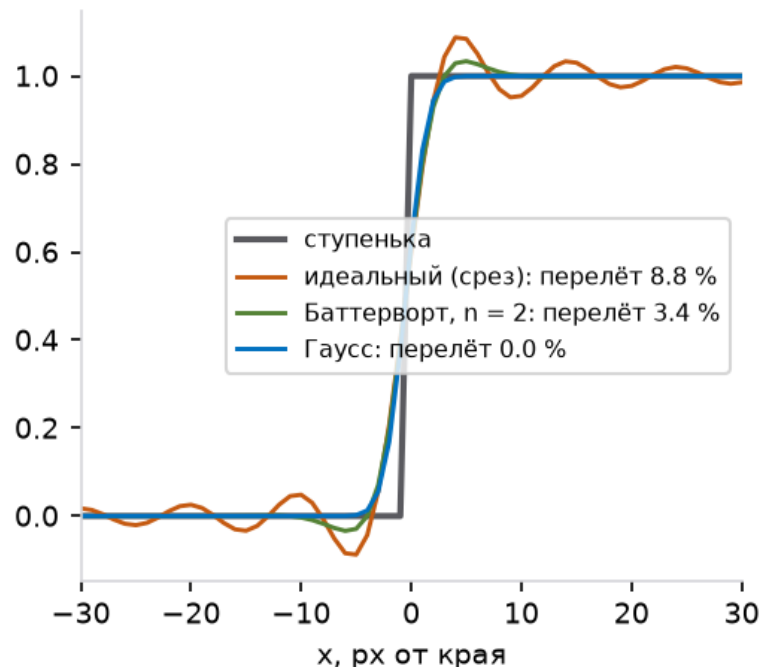
Маска умножается на спектр. Фильтр низких частот оставляет плавные перепады — в них 86 % энергии кадра. Фильтр высоких частот равен $1 - \text{ФНЧ}$: в нём 9 % энергии, половина значений отрицательна. Полосовой — разность двух Гауссов: детали одного масштаба. Режекторный гасит две точки и больше ничего не трогает

Произвольные маски фильтров можно конструировать непосредственно в частотной области и применять их путем прямого попиксельного умножения на спектр изображения, получая полный контроль над сохранением или удалением конкретных масштабов и частотных помех

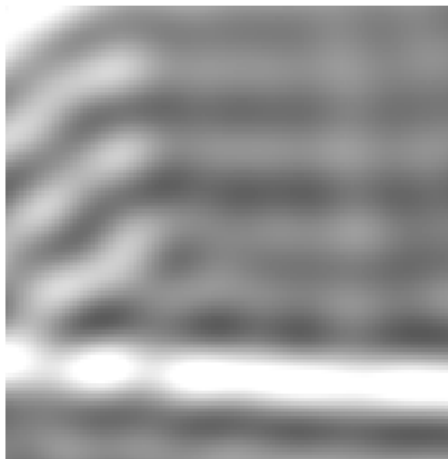
© фото — здание Университета Иннополис; собственная иллюстрация

Идеальный фильтр звенит

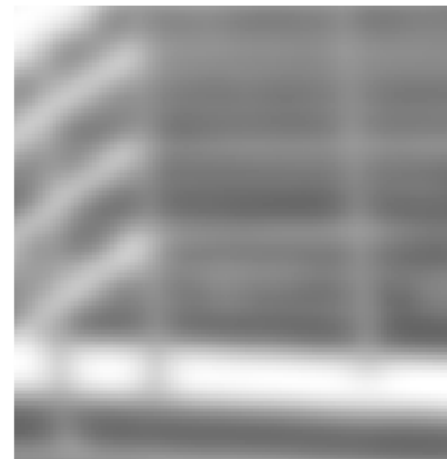
три ФНЧ с одной шириной $d_0 = 0.1$:
фронт одинаков, перелёт — нет



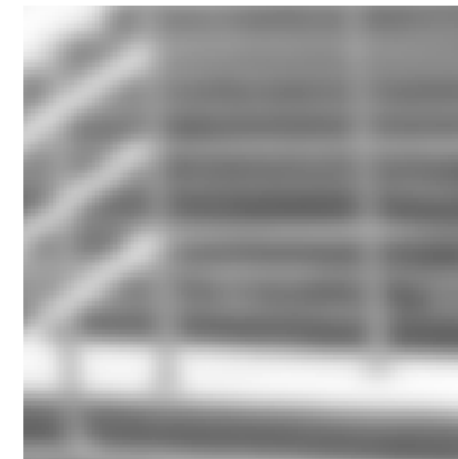
идеальный (срез), $d_0 = 0.05$:
21.4 дБ, значения -14...293



Баттерворт, $n = 2$, $d_0 = 0.05$:
22.1 дБ, значения -2...263



Гаусс, $d_0 = 0.05$:
22.6 дБ, значения 1...255



Резкий срез в частотах — это sinc в пространстве: вокруг каждого края появляются кольца. Перелёт идеального фильтра — 9 % скачка при любой ширине, у Баттерворта второго порядка — 3.4 %, у Гаусса — 0. На фото при $d_0 = 0.05$ идеальный фильтр проигрывает Гауссу 1.2 дБ и выходит за диапазон байта: значения от -14 до 293

«Идеальный» частотный фильтр с абсолютно резкой границей среза на практике оказывается худшим решением, поскольку он вызывает сильнейшие паразитные искажения («звон») вокруг контуров и портит значения пикселей, проигрывая по качеству более плавным аналогам — фильтрам Баттерворта и Гаусса

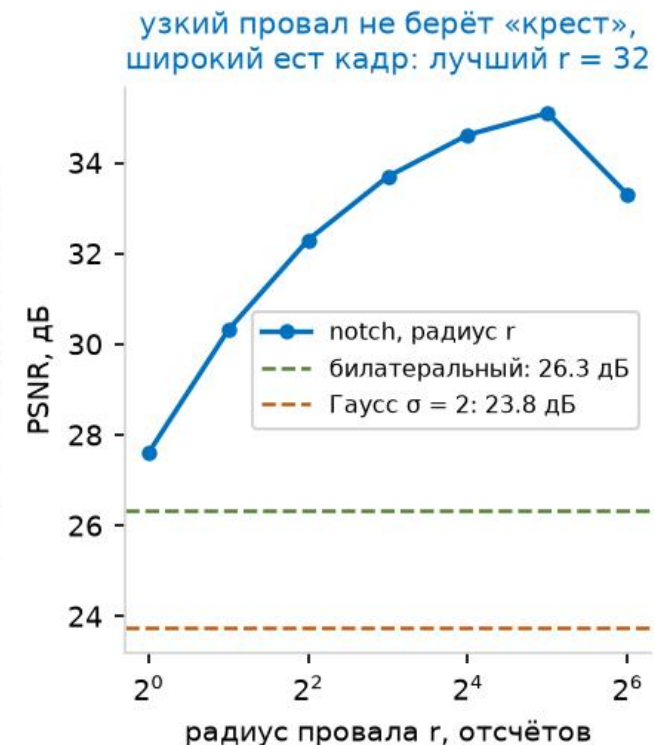
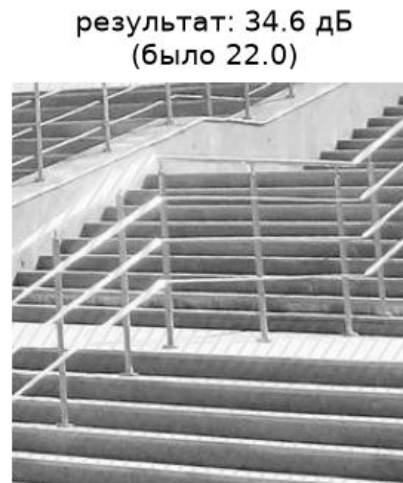
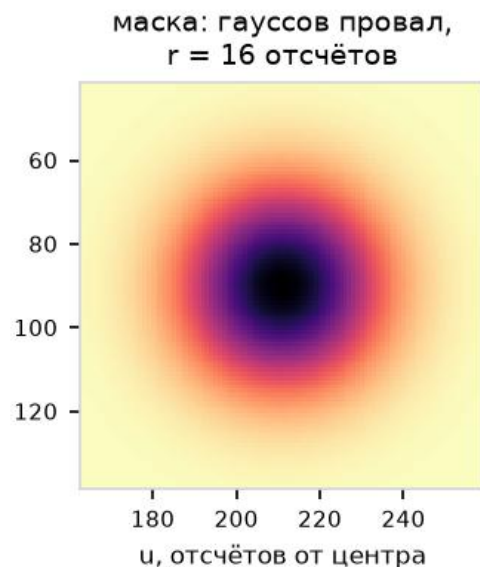
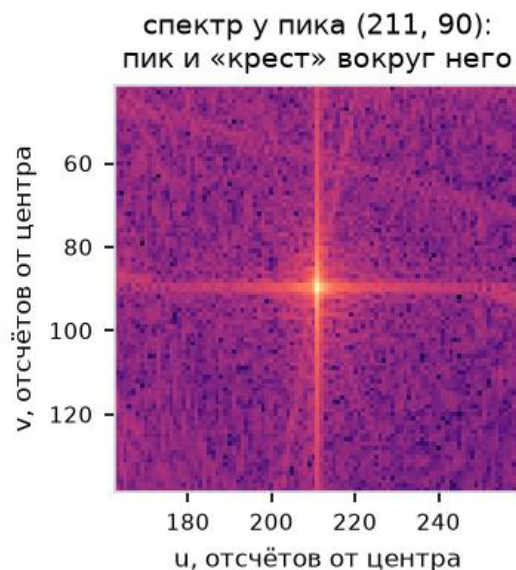
© фото — здание Университета Иннополис; собственная иллюстрация

Параметр	Смысл	Типичное	Что если больше
d_0 — частота среза, циклов на пиксель	граница полосы пропускания; у Гаусса $\sigma = 1 / (2\pi d_0)$ px	0.05–0.2; d_0 0.1 — это σ 1.59 px	кадр резче, шума больше; при 0.5 фильтр ничего не делает
форма маски	идеальная, Баттерворт порядка n , Гаусс	Гаусс	резче срез — сильнее кольца: $0 \rightarrow 3.4 \rightarrow 9\%$
r — радиус провала notch, отсчётов	сколько спектра вокруг пика гасим	8–32	$2 \rightarrow 30.3 \text{ дБ} \cdot 16 \rightarrow 34.6 \cdot 32 \rightarrow 35.1 \cdot 64 \rightarrow 33.3$
положение пика (du, dv)	частота помехи: период $1 / \sqrt{((du/W)^2 + (dv/H)^2)}$	ищется по спектру	гасить пик и его зеркальную пару ($-du, -dv$)

Гауссова маска с d_0 совпадает с GaussianBlur с σ до 0.001 кода — это один и тот же фильтр

- Частота среза d_0 : Задаёт границу полосы пропускания (для Гаусса жестко связана с пространственным сглаживанием формулой $\sigma = \frac{1}{(2\pi d_0)}$).
- Форма маски: Определяет закон затухания частот (идеальный срез, Баттерворт порядка n или Гаусс).
- Радиус провала режекторного фильтра (r): Показывает, сколько отсчетов спектра вокруг пика помехи необходимо заглушить.
- Положение пика помехи (du, dv): Точные координаты частоты шума в спектре. При этом нужно гасить также ($-du, -dv$)

Notch: помеха в двух точках



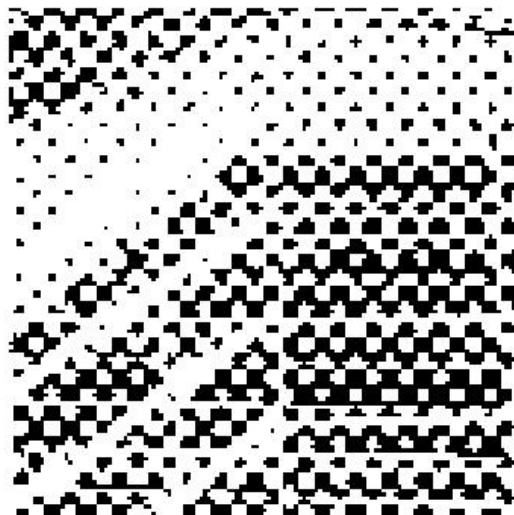
Пик помехи в 2471 раз выше фона; в чистом кадре на этом месте — 3.9 фона. Частота помехи не кратна размеру кадра, и пик расплылся в «крест»: провал радиусом 2 отсчёта даёт 30.3 дБ, 16 — 34.6, 32 — 35.1, 64 — уже 33.3, он начинает есть кадр. Гаусс и билатеральный фильтр остаются на 23.8 и 26.3 дБ

Из-за некрatности частоты помехи размеру кадра её пик в спектре расплывается в «крест», поэтому для идеальной очистки изображения радиус провала режекторного (notch) фильтра должен быть оптимального размера (лучший $r = 32$), чтобы захватить этот крест, но не начать вырезать полезные частоты самого кадра

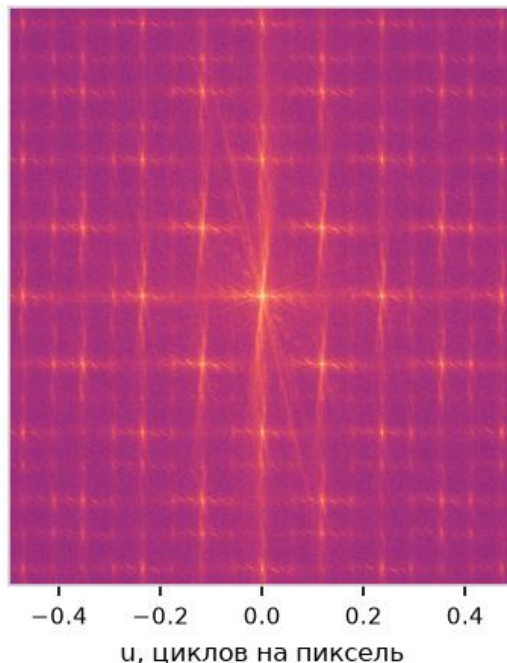
© фото — здание Университета Иннополис; собственная иллюстрация

Измерение: бинарный растр против notch

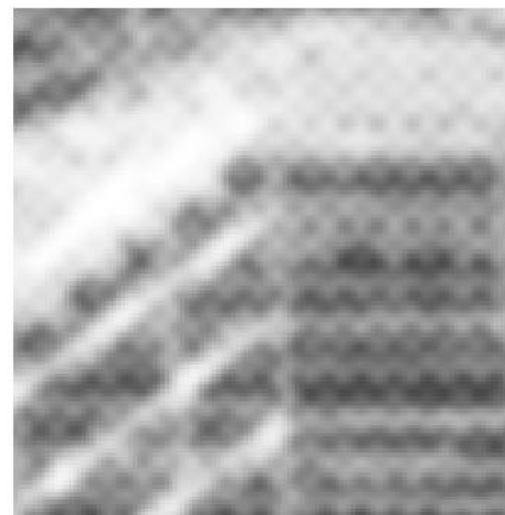
растр: чёрно-белые точки, шаг 6 px
(кроп 128 × 128): 8.5 дБ



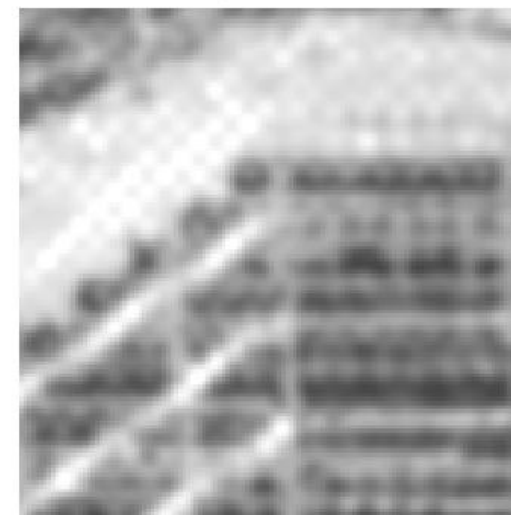
спектр: решётка из 48 пиков,
а не две точки



Гаусс $\sigma = 2.25$: 21.6 дБ



notch по решётке + ФНЧ: 21.8 дБ —
выигрыш 0.2 дБ



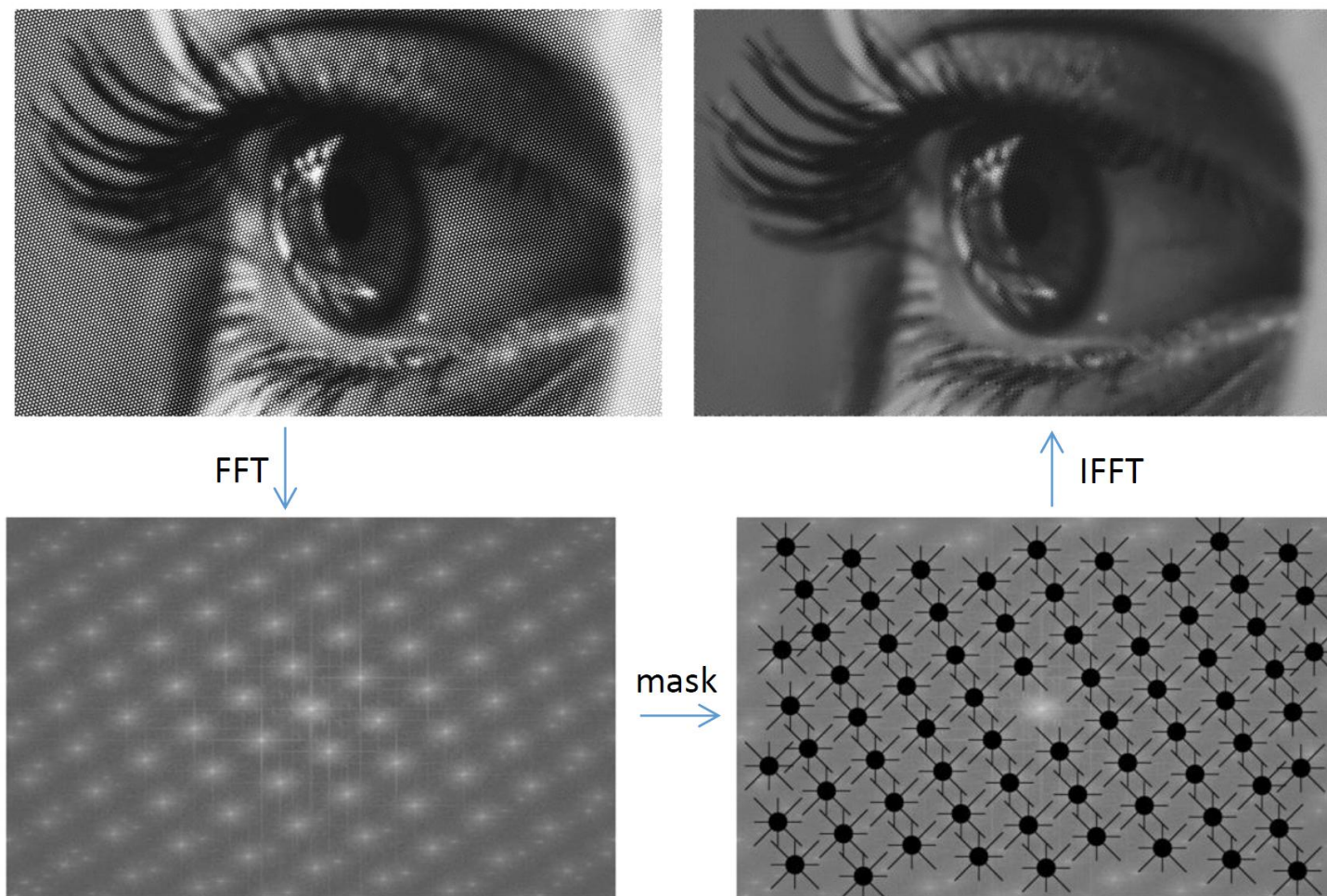
Правило «растр убирают notch-фильтром» проверено на чёрно-белом растре с шагом 6 px. В спектре не две точки, а решётка из 48 пиков; погасили все и добавили фильтр низких частот — 21.8 дБ. Обычный Гаусс $\sigma = 2.25$ — 21.6.

Выигрыш 0.2 дБ: растр — не прибавленная помеха, он уже уничтожил детали мельче своего шага

Против бинарного чёрно-белого растра частотная режекторная фильтрация (notch) неэффективна и практически бесполезна, поскольку такой шум является деструктивным (неаддитивным) преобразованием, которое уже необратимо уничтожило мелкие детали изображения

© фото — здание Университета Иннополис; собственная иллюстрация

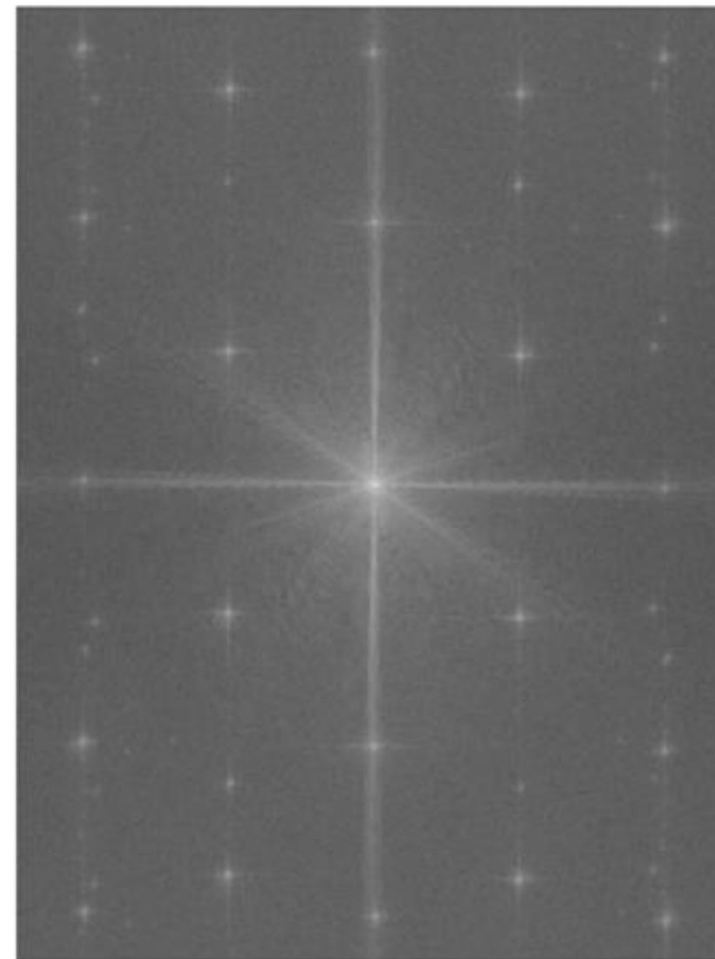
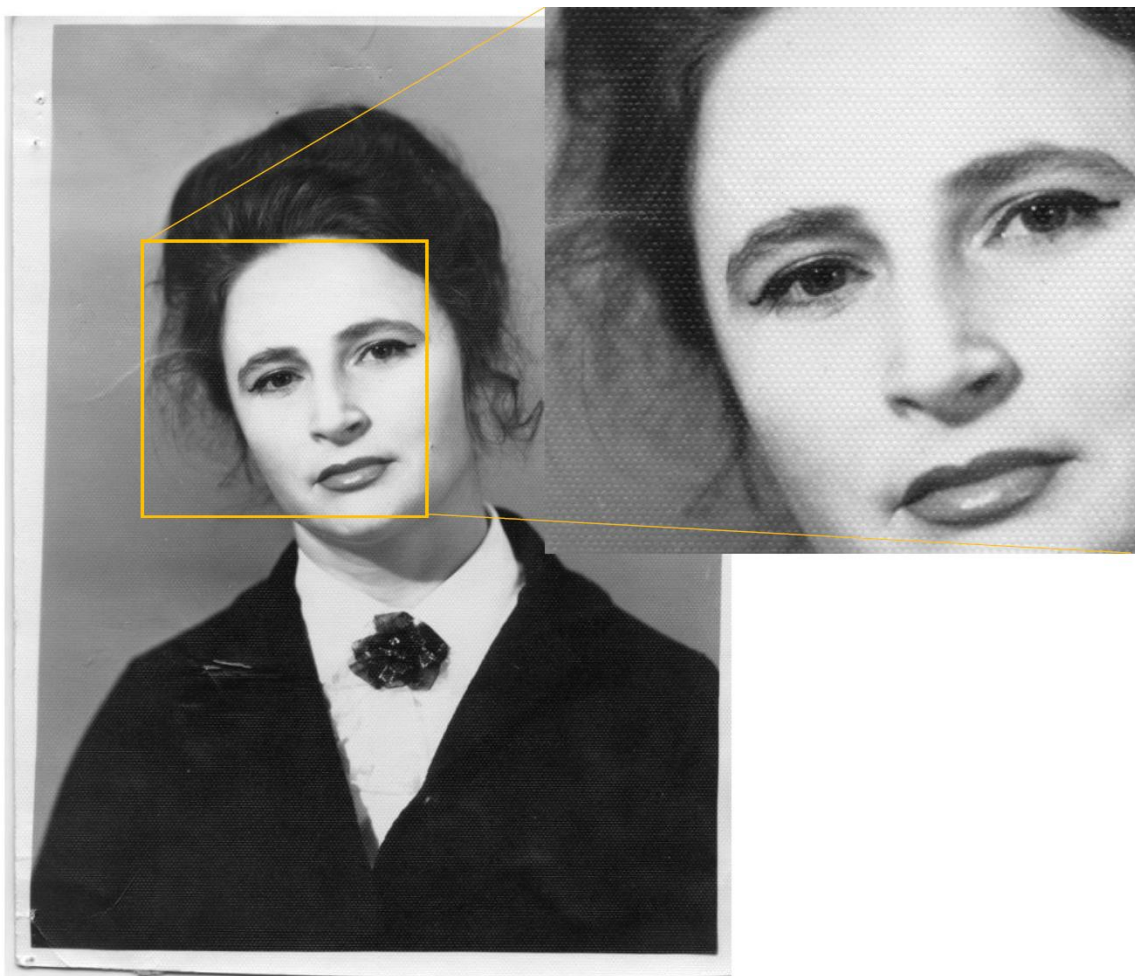
Удаление регулярного полутонового растра



Успешное практическое применение двумерного режекторного (notch) фильтра для полного удаления регулярного полутонового растра (Halftone) с целью восстановления гладкого и четкого изображения

© Fig. Dr. Oleg Perezyabov, Algorithms and Systems of Computer Vision, SPBPU, 2022

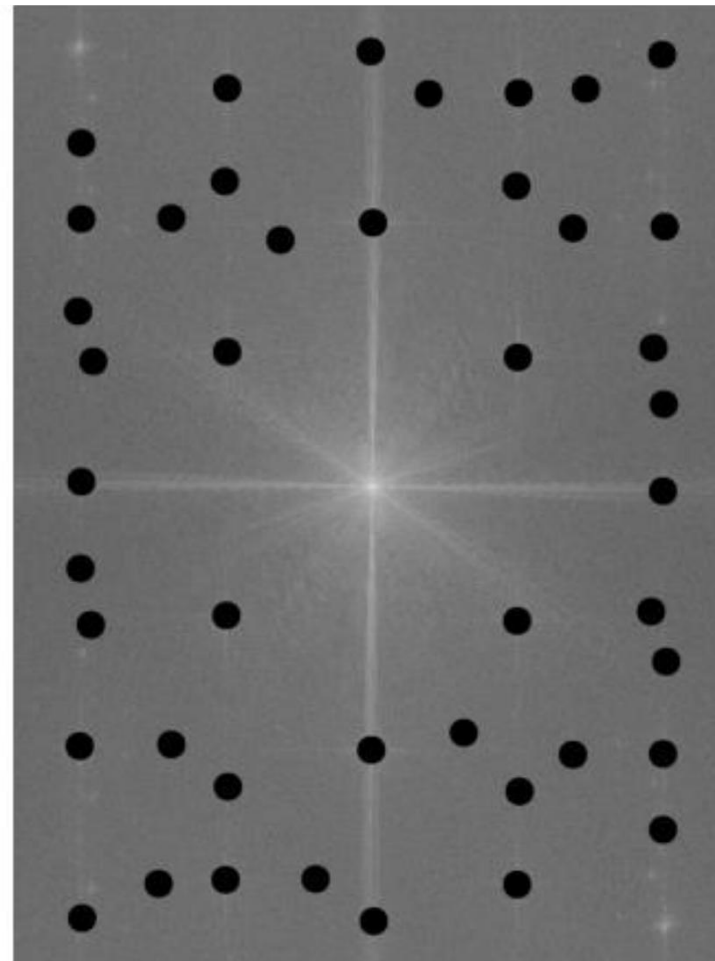
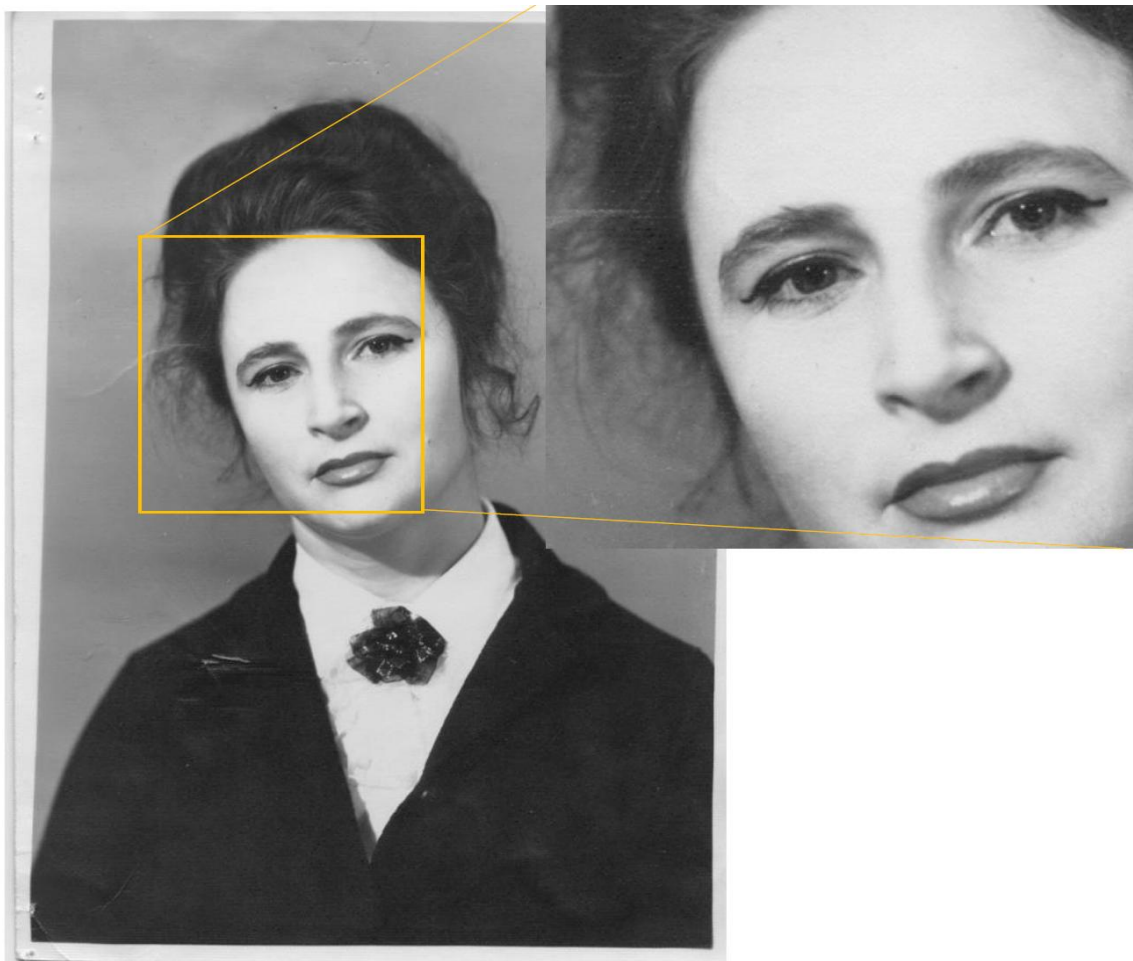
Удаление регулярного полутонного растра



Имеется старая фотография, напечатанная на текстурированной (тисненой) матовой бумаге или отсканированная через защитную сетку. Почему notch-фильтр здесь сработал идеально? Сама структура фотографии — полутонная (градации серого сохранялись).

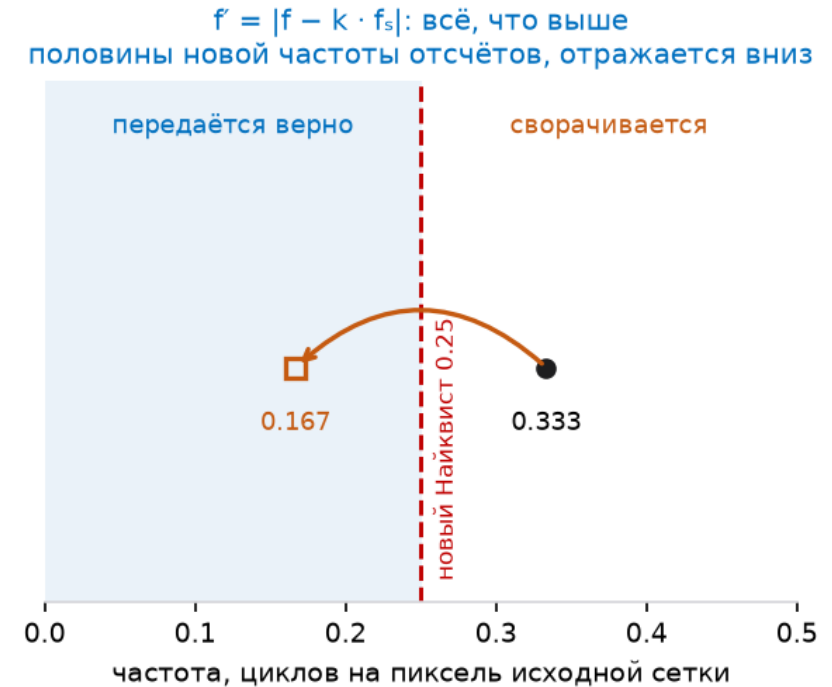
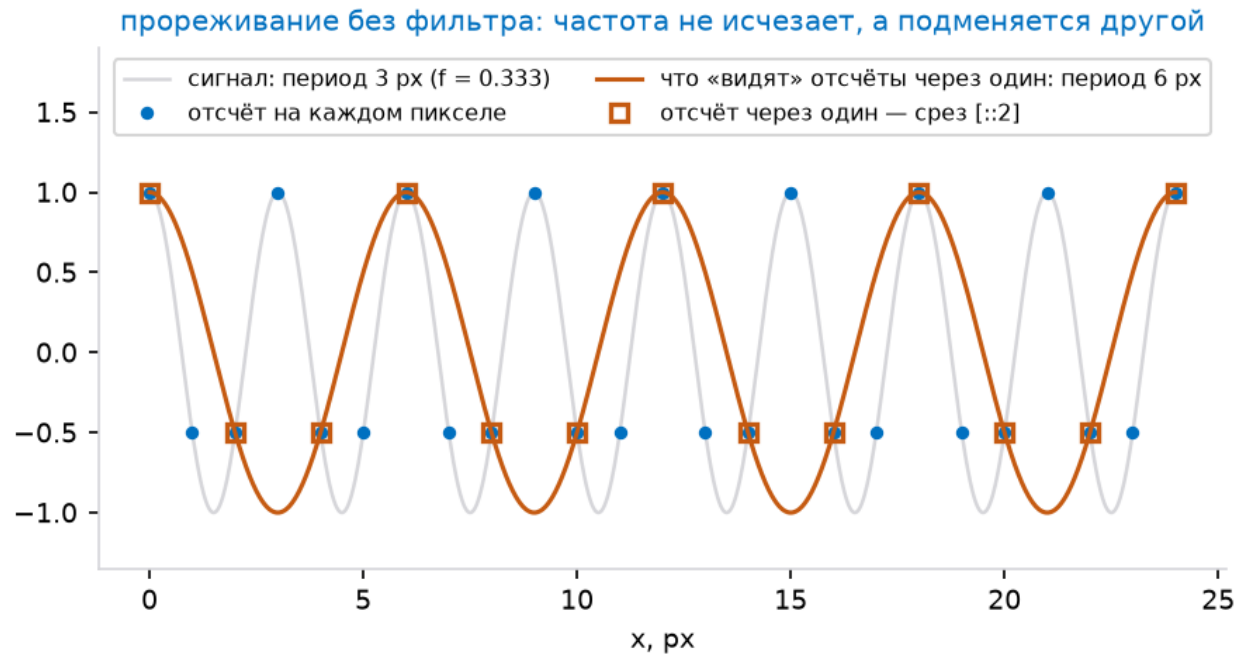
© Fig. Dr. Oleg Perezyabov, Algorithms and Systems of Computer Vision, SPBPU, 2022

Удаление регулярного полутонного растра



Мелкие детали лица (ресницы, поры, мягкие тени на коже) не были уничтожены, они просто смешались с наложенным поверх периодическим рельефом бумаги. По сути, эта регулярная текстура ведет себя как высокочастотный аддитивный узор

© Fig. Dr. Oleg Perezyabov, Algorithms and Systems of Computer Vision, SPBPU, 2022



Теорема Котельникова: сетка с шагом s передаёт частоты до $0.5 / s$ — это частота Найквиста. Всё, что выше, не исчезает, а отражается вниз: $f' = |f - k \cdot f_s|$. Решётка с периодом 3 px после среза каждого второго пикселя выглядит как решётка с периодом 6 px, период 2.5 px превращается в 10 px. Отличить подмену от настоящей детали после прореживания уже нельзя

Эффект алиасинга (подмены частот) при нарушении теоремы Котельникова/Найквиста: если уменьшить разрешение изображения (проредить пиксели) без предварительной фильтрации, высокие частоты не исчезают, а симметрично «отражаются» (сворачиваются) в область низких частот, порождая ложные крупные узоры.

© собственная иллюстрация

Алиасинг: четыре способа уменьшить

каждый 4-й пиксель
ложных колец 100 % · контраст 99 %

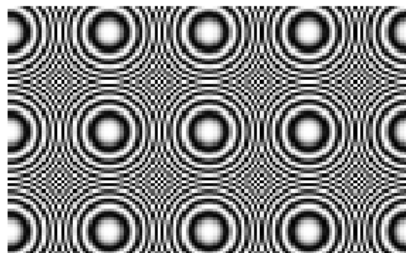
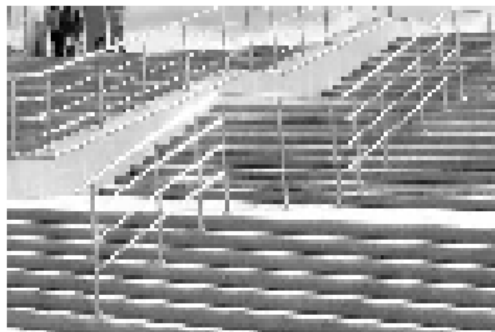


фото $\times 1/4$: 21.9 дБ



resize INTER_AREA
ложных колец 17 % · контраст 89 %

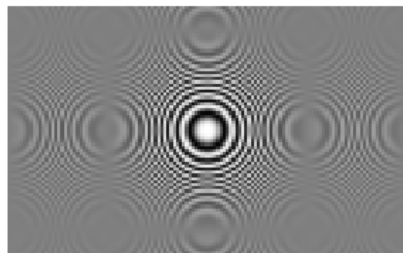


фото $\times 1/4$: 37.2 дБ



pyrDown $\times 2$
ложных колец 1 % · контраст 65 %

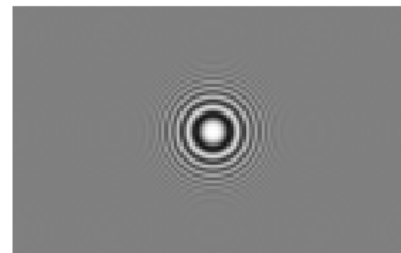


фото $\times 1/4$: 28.1 дБ (сетка сдвинута)



Гаусс $\sigma = 1.25$ + LINEAR
ложных колец 11 % · контраст 85 %

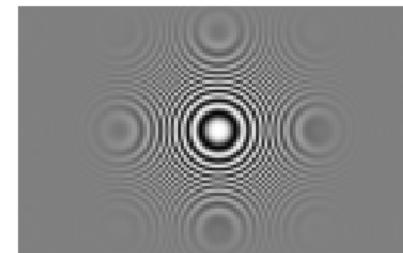


фото $\times 1/4$: 37.2 дБ

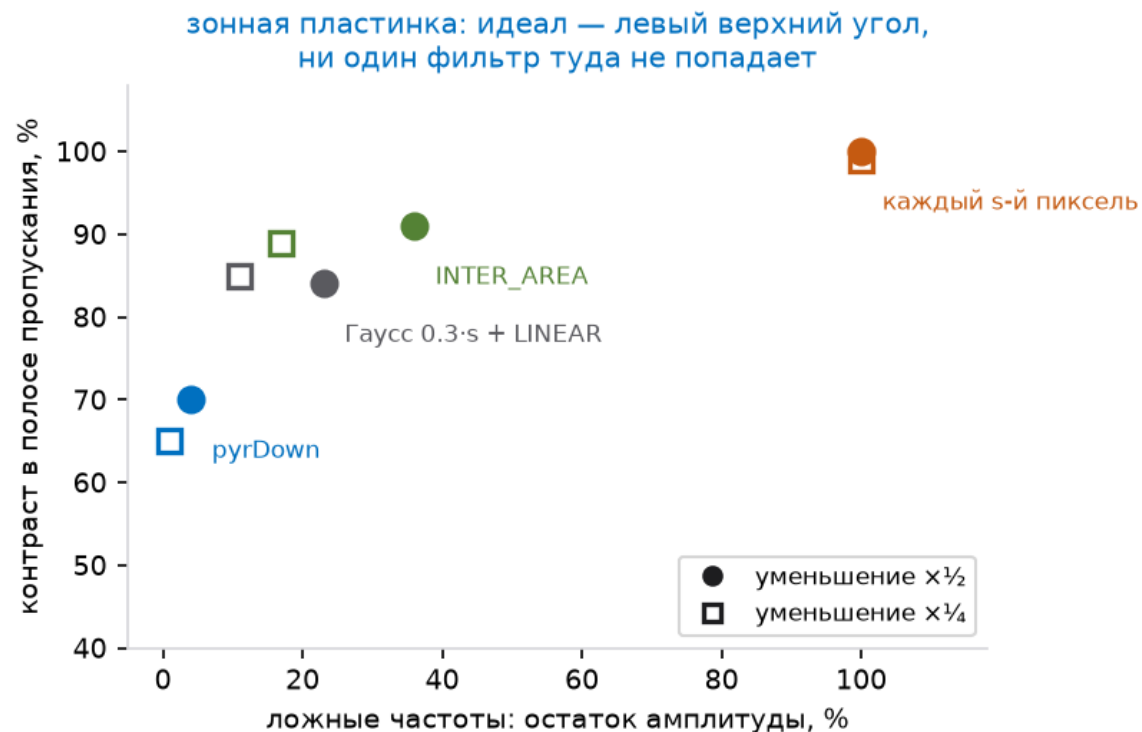
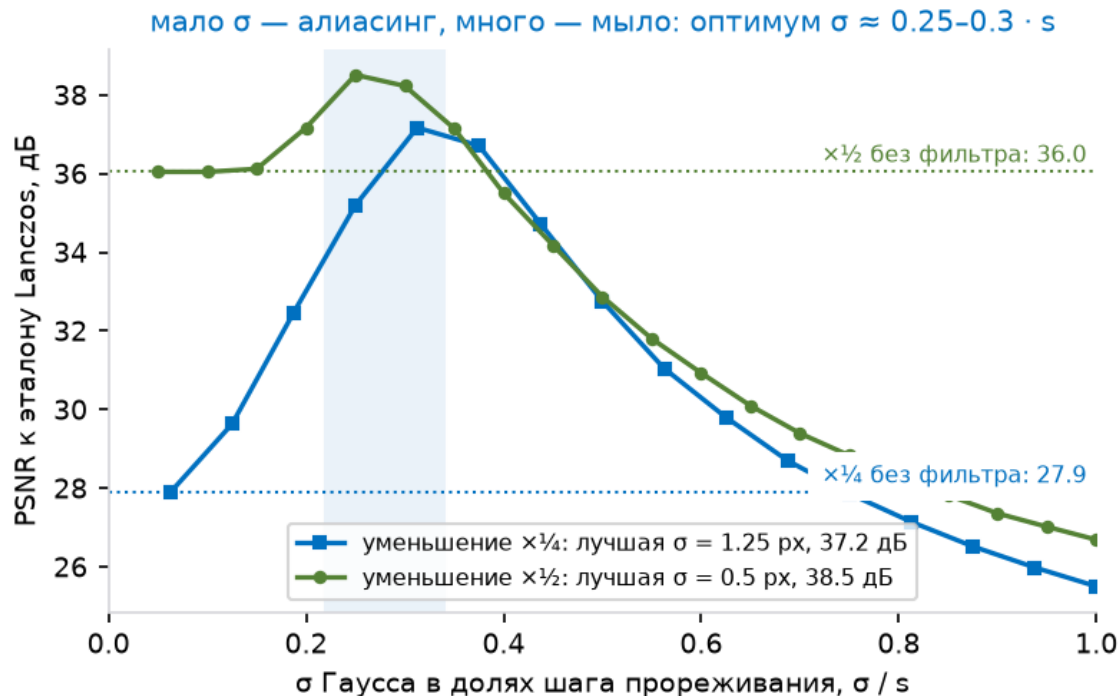


Зонная пластинка, частота растёт от центра к краю; уменьшаем $\times 1/4$. Без фильтра ложные кольца по всему кадру. INTER_AREA оставляет 17 % ложной амплитуды, Гаусс $\sigma 1.25$ — 11 %, pyrDown — 1 %, но контраст настоящих колец у него 65 % против 89. На фото 21.9 дБ без фильтра и 37.2 с фильтром

Качественный ресайз в компьютерном зрении — это всегда компромисс между подавлением ложного муара (чистотой) и сохранением резкости (контраста) реальных деталей

© фото — здание Университета Иннополис; собственная иллюстрация

Измерение: сколько размывать



Правило «перед уменьшением размыть» проверено по σ . Оптимум узкий: для $\times 1/4$ — $\sigma = 1.25 \text{ px}$ (37.2 дБ), $\sigma = 1.5$ из L4 — 36.7, $\sigma = 3$ — 27.9, как без фильтра; для $\times 1/2$ — $\sigma = 0.5 \text{ px}$. Правило: $\sigma \approx 0.25-0.3$ шага. Причина в спектре: выше нового Найквиста у кадра лежит 3.6 % энергии, 14.5 кода; без фильтра они ложатся на кадр как шум

Оптимальное математическое правило для размытия изображения перед его уменьшением: значение σ фильтра Гаусса должно составлять строго $[0.25 \dots 0.30]$ от шага прореживания s

© фото — здание Университета Иннополис; собственная иллюстрация

Операция на 1920 × 1279	Время	Замечание
<code>np.fft.fft2, float64</code>	130–150 мс	на 1920 × 1280 — 95–105: выигрыш ×1.4, а не ×4
<code>cv2.dft, float32</code>	20–22 мс	на оптимальном размере — 16; <code>getOptimalDFTSize(1279) = 1280</code>
свёртка через спектр, <code>cv2.dft</code>	≈ 50 мс	одинаково при ядре 5 × 5 и 101 × 101
<code>filter2D · sepFilter2D</code>	1.5–26 · 0.7–5 мс	ступенька <code>filter2D</code> между 11 и 13: 4 → 13 мс
<code>pyrDown</code> · пирамида Лапласа, 5 уровней	0.1–0.3 · 9 мс	обратная сборка — 8 мс
склейка по полосам, 6 уровней, цвет	≈ 190 мс	линейка — 76 мс
<code>matchTemplate</code> · по пирамиде, 3 уровня	35 · 0.6 мс	быстрее в 60 раз

Тайминги плавают от машины к машине — на слайде порядок величины

Для фильтрации маленькими ядрами и простых операций всегда следует использовать встроенные пространственные методы (`filter2D`, `pyrDown`), а к двумерному преобразованию Фурье (`cv2.dft`) переходить только при работе с большими масками, регулярными текстурными помехами или для фазовой корреляции, обязательно приводя стороны кадра к оптимальным степеням двойки/тройки/пятерки.

- **Антиалиасинг внутри сети:** свёрточная сеть уменьшает карту признаков шагом 2 без фильтра — ответ меняется от сдвига кадра на пиксель; BlurPool (Zhang, 2019) ставит перед шагом ядро $\begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix}$ или $\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \end{bmatrix}$ — это pyrDown
- **Пирамида признаков:** FPN (2017) и U-Net (2015) — пирамида Гаусса из карт признаков с «лапласовыми» добавками по пропускам; объекты разного размера ищутся на разных уровнях
- **Частоты как вход:** позиционное кодирование в трансформерах и NeRF — синусы и косинусы растущих частот
- **Где классика остаётся:** JPEG (DCT 8×8 , 1992) — основной формат снимков; `resize` с антиалиасингом; склейка по полосам в `cv2.Stitcher`; пирамиды в ORB (8 уровней) и в Лукасе–Канаде

Классические концепции частотного анализа, пирамид и антиалиасинга не устарели, а трансформировались в фундаментальные архитектурные слои современных нейросетей (архитектур 2026 года), связав классическое компьютерное зрение с глубоким обучением (Deep Learning)

2

Пирамиды

- Кадр в нескольких масштабах · пирамида Гаусса · пирамида Лапласа
- склейка по полосам · поиск от грубого к точному · вейвлеты

- **Задача 1 (Бесшовная склейка панорам):**

- *Дано:* Два кадра с перекрытием **480 px**, при этом правый кадр темнее на **24 кода** яркости и смещен с ошибкой в **3 px**.
- *Цель:* Склеить их в один кадр без видимого шва и без двоения объектов.

- **Задача 2 (Быстрый поиск по шаблону):**

- *Дано:* Маленький шаблон 96×96 и большой Full HD кадр 1920×1279 .
- *Цель:* Найти положение шаблона на кадре — быстрее, чем за **35 мс** (с минимальной ошибкой в пикселях).

- **Что объединяет эти задачи:**

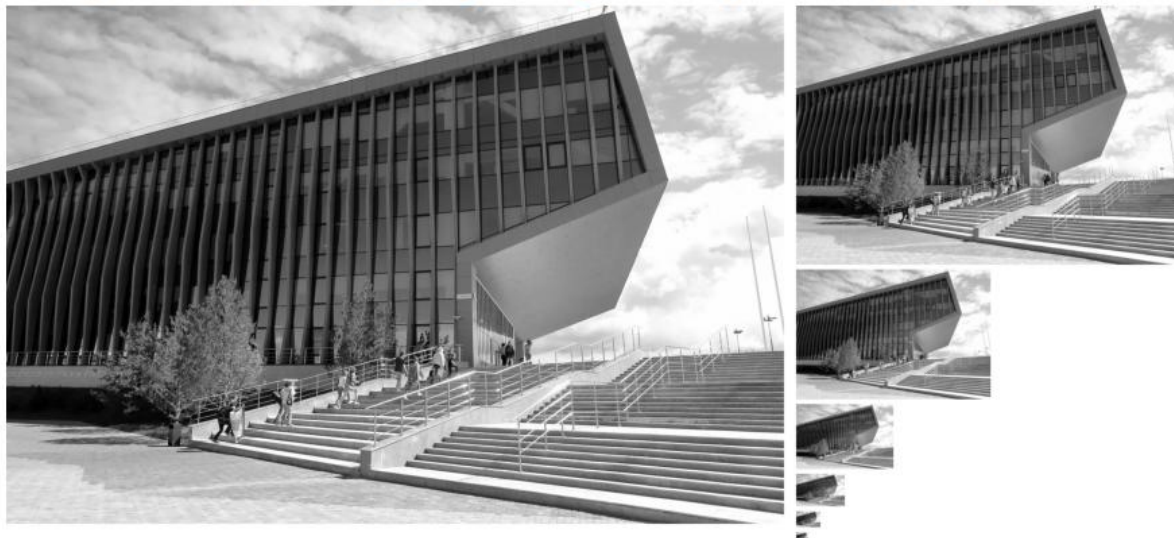
- Крупные и мелкие детали изображения требуют принципиально разного математического обращения в одной и той же точке кадра. Крупные детали (разница в яркости панорамы) нужно сглаживать на большом радиусе, а мелкие (геометрический сдвиг на 3 px) — сопоставлять на узком.

- **Резкий срез по шву:** Из-за перепада в 24 кода шов мгновенно бросается в глаза с любого расстояния в виде четкой линии.
- **Узкое линейное размытие (feathering 40 px):** Градиент яркости растягивается всего на 33 пикселя, из-за чего стык всё еще выглядит как грязная полоса, но при этом резкость у шва уже падает до **84%**.
- **Широкое линейное размытие (feathering 400 px):** Шов по яркости исчезает (переход 321 px), но из-за геометрической ошибки совмещения в 3 px кадры накладываются друг на друга в полупрозрачном режиме. Как итог — **контуры объектов делятся**, а резкость падает до катастрофических **45%**.
- **Классический полный перебор (полное сканирование кадра):** 35 мс на поиск одного шаблона одного масштаба; пять размеров — 175 мс, шесть кадров в секунду

У линейки одна ширина перехода на все частоты: яркости нужна широкая, деталям — узкая. Выбрать обе сразу она не может

Пирамида Гаусса

$G_0 1920 \times 1279 \rightarrow G_1 960 \times 640 \rightarrow G_2 480 \times 320 \rightarrow \dots \rightarrow G_6 30 \times 20$:
каждый шаг — размыть ядром $[1 \ 4 \ 6 \ 4 \ 1] / 16$ и выбросить каждый второй пиксель



Шаг пирамиды — размыть ядром $[1 \ 4 \ 6 \ 4 \ 1] / 16$ по каждой оси и выбросить каждый второй пиксель: `cv2.pyrDown`, доли миллисекунды. Семь уровней от 1920×1279 до 30×20 занимают вместе $4/3$ кадра. Ядро гасит частоту 0.5 до нуля, а на новой границе 0.25 оставляет четверть амплитуды: ложных частот 4 %, контраст 70 %

Эффективность пирамиды Гаусса (`cv2.pyrDown`) как компактного многомасштабного представления изображения: строится за доли миллисекунды, гарантирует минимальный уровень алиасинга (всего 4%) и занимает в памяти фиксированный объем, равный всей сумме уровней — строго $4/3$ от размера исходного кадра.

© фото — здание Университета Иннополис; собственная иллюстрация

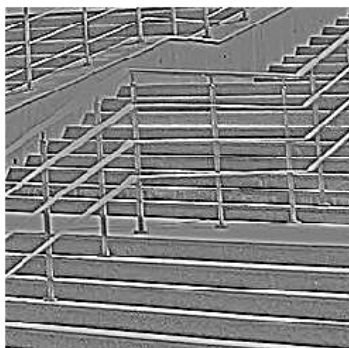
Пирамида Лапласа — спектр, который помнит место: полоса частот — по номеру уровня, положение — по пикселю

- **Уровень:** $L[k] = G[k] - \text{pyrUp}(G[k+1])$ — то, что потерялось при уменьшении: детали одной полосы частот; последний уровень — вершина пирамиды Гаусса
- **Что это значит физически:** кадр разложен по октавам — периоды 4, 9, 19, 37, 70 px; каждый уровень — полосовой фильтр, и каждый остаётся картинкой
- **Обратимо:** от вершины вниз — увеличить и прибавить; ошибка $8 \cdot 10^{-6}$ во float32
- **Значения со знаком:** в $L[0]$ 49 % отрицательных; `cv2.subtract` в uint8 их обнуляет — восстановление 18.9 дБ

Пирамида Лапласа разбивает изображение на полосы частот (октавы деталей) без потери информации, сохраняя ровно те данные, которые теряются при уменьшении кадра Гауссом, что позволяет полностью и безошибочно восстановить исходный снимок

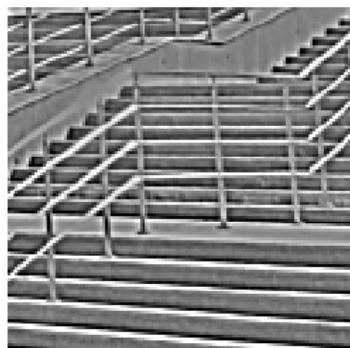
Пирамида Лапласа: уровни

L0: период ≈ 4 px,
55 % значений в ± 2



Уровень L_0 вырезает
только самые мелкие
детали с периодом 4px

L1: период ≈ 9 px,
47 % значений в ± 2



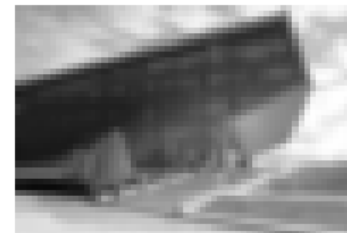
Уровень L_1 фиксирует
средние детали с
периодом 9px

L2: период ≈ 19 px,
35 % значений в ± 2



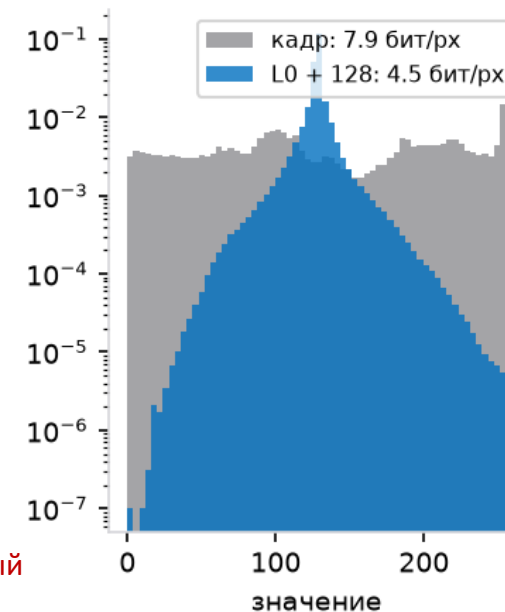
Уровень L_2 берёт
крупные контуры с
периодом 19px

вершина G_5 , 60×40 :
остаток низких частот



Вершина G_5 (60×40) - это
базовый «остаток низких
частот» (силуэт лестницы и
общее освещение), на который
при обратной сборке будут
наназываться эти контуры.

уровень — узкий пик у нуля:
его дёшево хранить

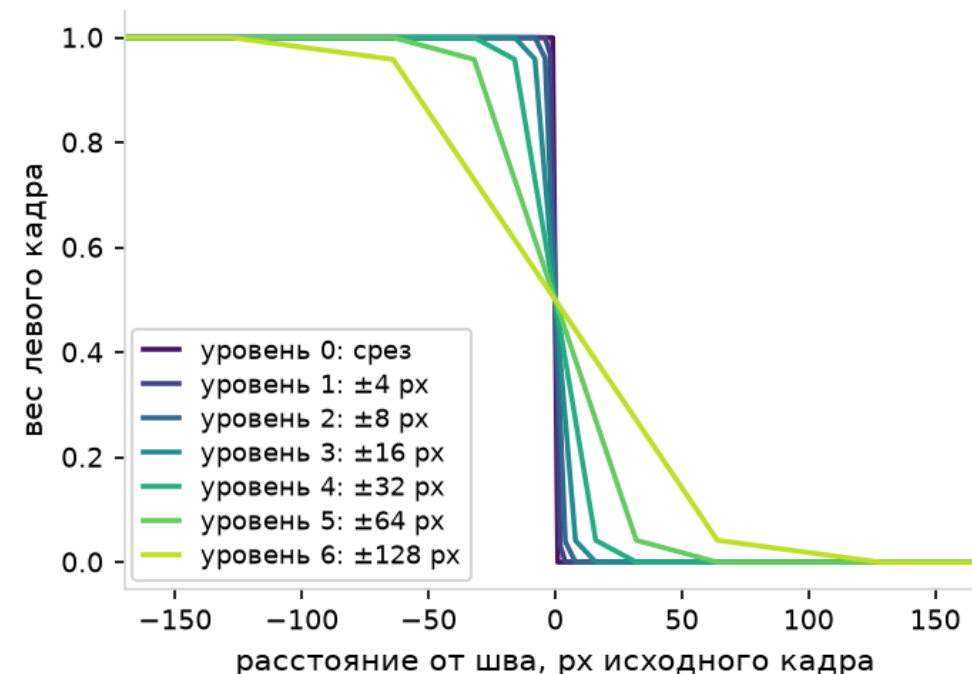


Уровни L_0 – L_2 на одном кропе: те же ступени, но каждый уровень держит детали своего размера. Значения сосредоточены у нуля: в L_0 55 % в пределах ± 2 , энтропия 4.5 бита на пиксель против 7.9 у кадра. Квантование двух нижних уровней с шагом 8 даёт 2.9 бита на пиксель при 41.2 дБ — пирамиду придумали в 1983 году как код сжатия

На глубоких уровнях пирамиды Лапласа хранятся не «размытые контуры», а строго отфильтрованные полосы геометрических масштабов, которые к тому же математически выгодно сжимать

© фото — здание Университета Иннополис; собственная иллюстрация

- **Идея Бёрта и Адельсона, 1983:** оба кадра — в пирамиды Лапласа, маску шва — в пирамиду Гаусса; смешать уровень с уровнем и собрать обратно
- $L[k] = M[k] \cdot LA[k] + (1 - M[k]) \cdot LB[k]$
- **Что это значит физически:** маска на уровне k размыта на $\pm 2^{k+1}$ px от шва — мелкие детали (уровень 1) смешиваются в зоне ± 4 px, перепады яркости (уровень 6) — в зоне ± 128 px
- **Каждая полоса — на ширине порядка своей длины волны:** двоиться нечему, шву негде появиться



Широкий переход для яркости и узкий для деталей — одновременно: то, чего не могла линейка

Классический алгоритм мультимасштабной бесшовной склейки изображений (Pyramid Blending) Бёрта и Адельсона идеально решает проблему стыков панорам, адаптируя ширину зоны смешивания под каждый частотный диапазон индивидуально

© собственная иллюстрация

Склейка: четыре способа

два кадра на общем холсте, перекрытие 480 px:
правый темнее на 24 кодов, сдвинут на 3 px



срез по шву:
переход 1 px,
резкость 99 %



линейка 40 px:
переход 33 px,
резкость 84 %



линейка 400 px:
переход 321 px,
резкость 45 %



multi-band, 6 ур.:
переход 134 px,
резкость 98 %

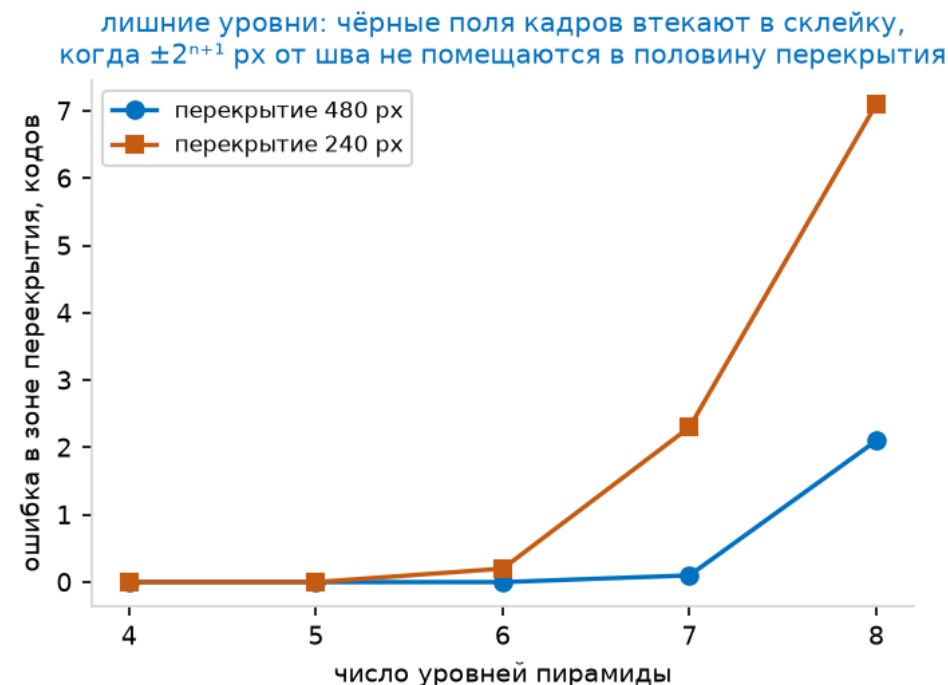
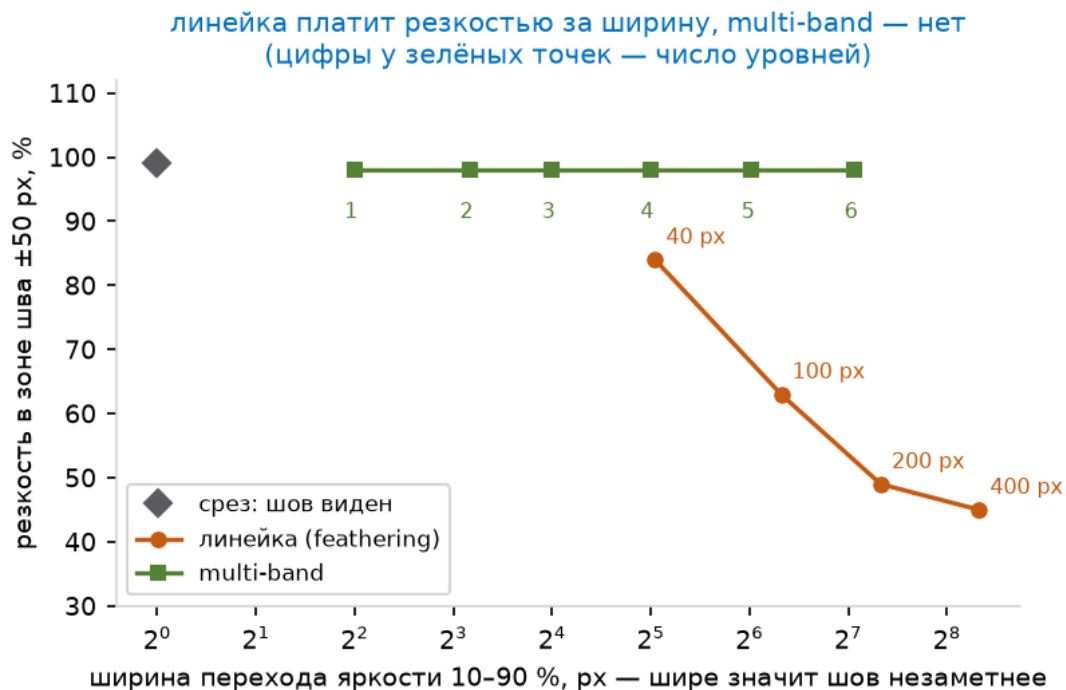


Один шов, четыре склейки. Срез: переход 1 px — шов виден. Линейка 40 px: полоса шириной 33 px, резкость 84 %. Линейка 400 px: шва нет, но резкость 45 % — дерево и люди двоются. Склейка по полосам, 6 уровней: яркость переходит на 134 px при резкости 98 % — широкий переход для яркости и узкий для деталей одновременно

Классический алгоритм мультимасштабной бесшовной склейки изображений (Pyramid Blending) Бёрта и Адельсона идеально решает проблему стыков панорам: он распределяет перепад яркости на 134 пикселя (широкий невидимый переход для низких частот), но удерживает резкие контуры деталей в ультраузком коридоре, сохраняя колоссальные 98% резкости и полностью исключая двоение объектов.

© фото — здание Университета Иннополис; собственная иллюстрация

Измерение: сколько уровней



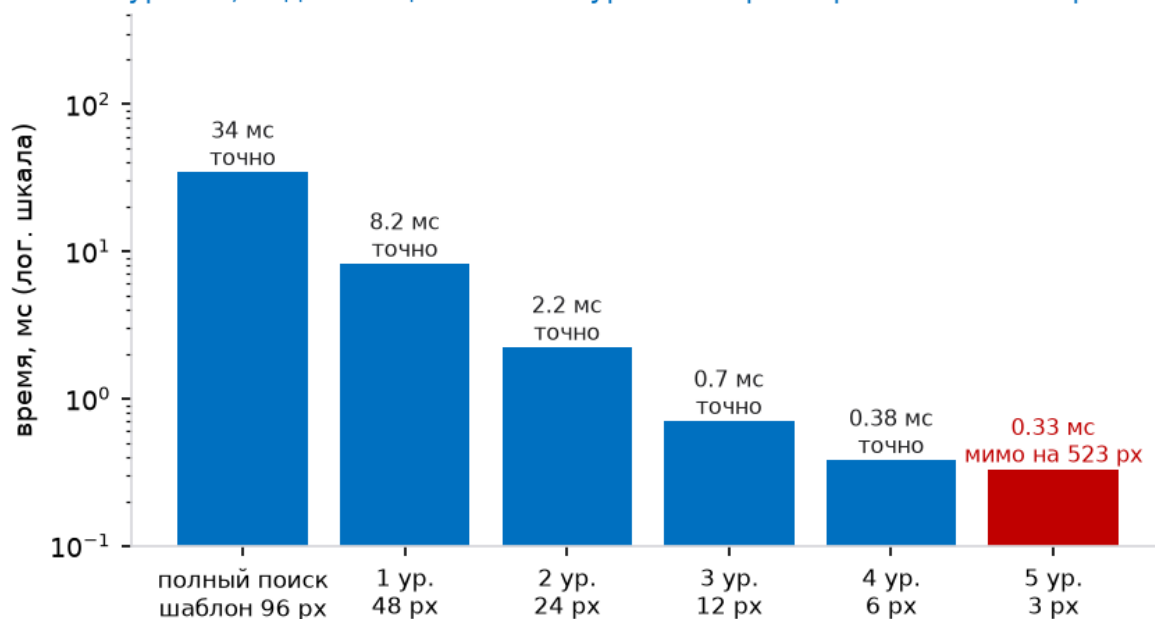
Правило из tutorиалов «берите шесть уровней» проверено. Каждый уровень удваивает ширину перехода яркости: 4, 9, 16, 32, 65, 134 px; резкость 98 % при любом числе уровней. Предел ставит перекрытие: зона смешивания $\pm 2^{n+1}$ px от шва должна помещаться в его половину, иначе в склейку втекают чёрные поля кадров — при перекрытии 240 px и восьми уровнях ошибка 7 кодов

Глубина пирамиды лимитирована шириной перекрытия кадров, превышение которой приводит к втеканию черных полей в зону шва и резкому росту ошибок

© собственная иллюстрация

Поиск от грубого к точному

matchTemplate по пирамиде: грубо на вершине, уточнение ± 2 px на каждом уровне; под столбцом — число уровней и размер шаблона на вершине



уровень 3: кадр 240 × 160, шаблон 12 × 12 —
пик 0.992, второй 0.832



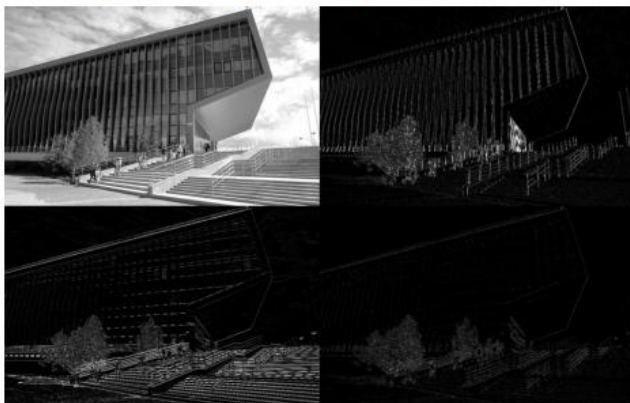
Шаблон ищется на вершине пирамиды, затем координата удваивается и уточняется в окне ± 2 px на каждом уровне.

Три уровня: 0.6 мс вместо 35, ответ тот же. Пять уровней: шаблон на вершине 3 × 3, пик 0.993 при втором 0.992 — промах на 523 px. Правило: шаблон на вершине не меньше 8–12 px

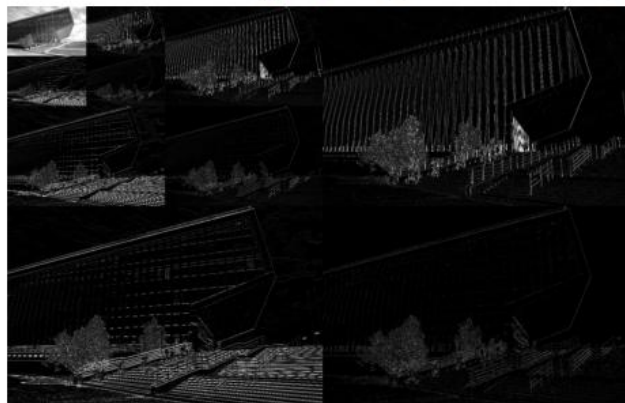
При проектировании многомасштабного трекинга или сопоставления шаблонов нельзя бездумно увеличивать число уровней пирамиды. Количество слоев должно строго ограничиваться снизу: минимальный размер шаблона на самом верхнем уровне пирамиды обязан оставаться в пределах 8–12 пикселей, чтобы избежать потери структуры и ложного ухода алгоритма в сторону.

© фото — здание Университета Иннополис; собственная иллюстрация

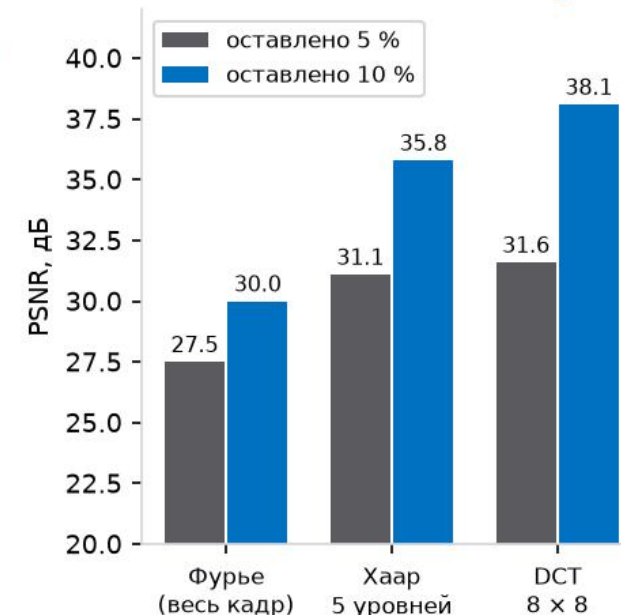
один шаг Хаара: LL — уменьшенный кадр,
LH, HL, HH — детали трёх ориентаций



три шага: коэффициентов ровно столько,
сколько пикселей (у пирамиды Лапласа — 4/3)



локальный базис сжимает лучше



Шаг вейвлета Хаара — среднее и разность соседей по каждой оси: четверть LL и три четверти деталей по ориентациям. Коэффициентов ровно столько, сколько пикселей; 74–89 % деталей лежат в пределах ± 2 . Оставили 10 % коэффициентов: Фурье всего кадра — 30.0 дБ, Хаар — 35.8, DCT блоков 8×8 — 38.1: локальный базис выигрывает. Отсюда JPEG и JPEG 2000

Локальный базис (вейвлеты и DCT блоками 8×8) сжимает изображения эффективнее глобального Фурье-базиса, поскольку коэффициенты вейвлет-преобразования не увеличивают избыточность в памяти и сохраняют локальные пространственные особенности кадра

© фото — здание Университета Иннополис; собственная иллюстрация

- **Динамические объекты на панорамах (Эффект «призрака»):** Если объект (человек) успел переместиться между кадрами, классическая склейка по полосам Бёрта-Адельсона не сотрет его, а отобразит в полупрозрачном виде в обоих местах.
- **Физика падения резкости при сдвиге:** Ошибка совмещения кадров всего в **3 px** незаметна на глубоких уровнях пирамиды (где период деталей 19 px), но на самом верхнем уровне (период **4 px**) контуры уже не смешиваются плавно, а стыкуются встык. Именно эта физическая граница не позволяет получить 100% идеальную резкость, ограничивая её на уровне **98%**.
- **Опасность циклических границ (BORDER_WRAP)** - При использовании быстрого преобразования Фурье алгоритмы OpenCV по умолчанию зацикливают края кадра.
- **Артефакты фазовой корреляции:** `cv2.phaseCorrelate` умножает входные кадры на оконную функцию (окно Ханна/Хемминга). Если размер изображения не оптимизирован под DFT, операция заваливает края: на кадрах 960 × 640 изображение портится, тогда как на честном Full HD 1920 × 1279 этого не происходит.

Признак беды — всегда число: второй пик рядом с первым, значения за пределами 0...255, ошибка у края больше, чем в центре

В частотной области

- помеха периодическая и прибавлена к кадру: notch, 22.0 $\rightarrow$ 34.6 дБ
- надо понять, что фильтр делает: смотреть спектр ядра
- ядро большое и несепарабельное: $\approx$ 50 мс при любом размере
- сдвиг между кадрами: фазовая корреляция, 10 мс

В пространстве

- ядро маленькое или сепарабельное: 0.7–5 мс против 50
- фильтр зависит от места или от данных: билатеральный, медиана, адаптивный порог
- помеха не периодическая: в спектре ей не за что зацепиться
- у кадра резкие края поля: спектр «видит» разрыв на границе

- **Помеха не попадает в сетку частот:** пик расплывается в «крест», провал нужен шире одной точки — $r = 2$ даёт 30.3 дБ, $r = 16$ — 34.6
- **Сигнал упёрся в край диапазона:** у 9 % пикселей помеха перестала быть синусоидой; даже точное вычитание известной помехи даёт 34.1 дБ
- **Помеха умножена, а не прибавлена** — мерцание ламп $\pm 25\%$ при rolling shutter, L2: notch даёт 28.4 дБ, точное деление на известный профиль — 29.8
- **Растр и сильное сжатие уже уничтожили детали:** фильтр их не вернёт — выигрыш notch на растре 0.2 дБ

Notch лечит то, что прибавлено и узкополосно. Остальное — вопрос к модели помехи, а не к фильтру

- **Спектр:** `float64` $\rightarrow$ `fft2` $\rightarrow$ `fftshift`; смотреть $\log(1 + |F|)$; цветной кадр — по каналам или `axes=(0, 1)`
- **Фильтр в частотах:** маска гладкая и симметричная относительно центра; обратно — `ifftshift` $\rightarrow$ `ifft2` $\rightarrow$ `.real` $\rightarrow$ `np.clip`
- **Помеха:** закрыть центр спектра, `argmax` — пик; провал $r = 8-32$ на пике и на зеркальной паре
- **Уменьшение:** `resize(..., INTER_AREA)`; в варпе — Гаусс $\sigma \approx 0.3$ шага до варпа; ровно вдвое и много раз — `pyrDown`
- **Пирамида:** `float32`, `dstsize` в `pyrUp`; уровней — пока шаблон на вершине не меньше 8 px, а 2^{n+1} px не больше половины перекрытия
- **Что проверить первым, если «странно»:** забытый `fftshift`; `np.abs` вместо `.real`; `uint8` в разности уровней

Рецепт повторяется в ta-guide занятия 6 — студент и ассистент видят одно и то же

Робототехника

- уменьшение кадра на борту без ложных узоров: решётки, заборы, плитка — INTER_AREA за 0.7 мс
- поиск от грубого к точному — в 60 раз быстрее; так устроен пирамидальный Лукас–Канаде (L13), а ORB (L9) ищет точки на 8 уровнях пирамиды
- фазовая корреляция: сдвиг между кадрами за 10 мс без единой точки интереса

МОИИ

- BlurPool — это pyrDown внутри сети: без фильтра ответ меняется от сдвига на пиксель
- FPN и U-Net — пирамиды признаков; аугментации: JPEG, размытие, уменьшение
- спектр — диагностика данных: растр, муар и следы сжатия видны как точки и сетки

- **Умеем:** построить пирамиды Гаусса и Лапласа и вернуться к кадру с ошибкой $8 \cdot 10^{-6}$; склеить два кадра — переход яркости 134 px при резкости 98 %; ускорить поиск шаблона в 60 раз и знать, где он промахнётся
- **Блок II закрыт:** геометрия меняет места пикселей (L4), фильтр — значения по окрестности (L5), спектр и пирамида объясняют, что при этом происходит с деталями разного размера (L6)
- **Открытый вопрос:** как из свёртки получить производную, а из производной — край и прямую: **L7** — Собель, Canny, преобразование Хафа и RANSAC

Демо 2.1 — уменьшение $\times \frac{1}{4}$: пять способов

```
1 from PIL import Image
2 size = (480, 320) #  $\times \frac{1}{4}$  от 1920 x 1279
3 ref = np.asarray(Image.fromarray(g).resize(size, Image.LANCZOS)) # эталон
4 rs = lambda im, flag: cv2.resize(im, size, interpolation=flag)
5 blur = cv2.GaussianBlur(g, (0, 0), 1.25) #  $\sigma \approx 0.3$  шага
6 ways = {"каждый 4-й пиксель": rs(g, cv2.INTER_NEAREST),
7         "INTER_LINEAR": rs(g, cv2.INTER_LINEAR),
8         "Гаусс  $\sigma$  1.25 + LINEAR": rs(blur, cv2.INTER_LINEAR),
9         "INTER_AREA": rs(g, cv2.INTER_AREA),
10        "pyrDown дважды": cv2.pyrDown(cv2.pyrDown(g))}
11 for name, im in ways.items():
12     print("%-22s %.1f дБ" % (name, psnr(ref, im)))
```

python

каждый 4-й пиксель 21.9 дБ · INTER_LINEAR
27.9 · Гаусс σ 1.25 + LINEAR 37.2
INTER_AREA 37.2 · pyrDown дважды 28.1
(сетка сдвинута на 1.5 px источника)

Вызов

`cv2.resize(im, size, interpolation=flag)`

`cv2.GaussianBlur(g, (0, 0), 1.25)`

`cv2.pyrDown(g)`

Что делает

NEAREST берёт один пиксель из 16, LINEAR — 4, AREA усредняет все 16

фильтр до прореживания: $\sigma \approx 0.3$ шага, найдено измерением

ядро $\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \end{bmatrix}$ / 16 и каждый второй пиксель; размер результата $((W + 1) // 2, (H + 1) // 2)$

Демо 2.2 — пирамида Лапласа

python

```
1 def gauss_pyr(x, n):                                # кадр и n уменьшенных копий
2     return [x] if n == 0 else [x] + gauss_pyr(cv2.pyrDown(x), n - 1)
3 def lap_pyr(x, n):                                   # разности уровней + вершина
4     gp = gauss_pyr(x, n)
5     return [a - cv2.pyrUp(b, dstsize=a.shape[1::-1])
6             for a, b in zip(gp, gp[1:])] + [gp[-1]]
7 def collapse(p):                                     # от вершины вниз
8     x = p[-1]
9     for l in p[-2::-1]: x = cv2.pyrUp(x, dstsize=l.shape[1::-1]) + l
10    return x
11 lp = lap_pyr(g.astype(np.float32), 5); neg = 100 * (lp[0] < 0).mean()
12 print("обратно: %.0e, в L0 меньше нуля %.0f %" % (abs(collapse(lp)) - g).max(), neg))
```

обратно: 8e-06, в L0 меньше нуля 49 %
та же пирамида в uint8 через
cv2.subtract: обратно 18.9 дБ · pyrUp без
dstsize: 1920 × 1280 при уровне 1920 ×
1279

Вызов

cv2.pyrUp(b, dstsize=a.shape[1::-1])

zip(gp, gp[1:])

p[-2::-1]

Что делает

увеличить вдвое до размера уровня ниже; shape[1::-1] —
(ширина, высота)

пары соседних уровней: (G0, G1), (G1, G2), ...

уровни от предпоследнего к нулевому — порядок сборки

Демо 2.3 — склейка: линейка против полос

python

```
1 A = cio.imread("demo/pano-left.jpg").astype(np.float32) # левый кадр
2 B = cio.imread("demo/pano-right.jpg").astype(np.float32) # правый кадр
3 H, W = A.shape[:2]; xs = W // 2; x = np.arange(W, dtype=np.float32) # xs — шов
4 def mask(w):
5     # вес левого кадра: линейка шириной w
6     return np.tile(np.clip(0.5 - (x - xs) / w, 0, 1)[None, :, None], (H, 1, 3))
7 alpha = lambda w: lambda b: A * mask(w) + b * (1 - mask(w)) # по маске
8 def bands(n):
9     # по полосам частот; функции демо 2.2
10    ps = lambda b: zip(lap_pyr(A, n), lap_pyr(b, n), gauss_pyr(mask(1e-6), n))
11    return lambda b: collapse([la * m + lb * (1 - m) for la, lb, m in ps(b)])
12 for name, fn in (("срез по шву", alpha(1e-6)), ("линейка 40 px", alpha(40)),
13                 ("линейка 400 px", alpha(400)), ("multi-band, 6 ур.", bands(6))):
14     seam_report(name, fn, A, B, xs) # переход яркости и резкость у шва
```

срез по шву: переход яркости 1 px · резкость у шва 99 % · линейка 40 px: 33 px · 84 %
линейка 400 px: 321 px · 45 % · multi-band, 6 ур.: 134 px · 98 % (значения от -22 до 255 — нужен `np.clip`)

Вызов

`np.clip(0.5 - (x - xs) / w, 0, 1)`

`[None, :, None] → np.tile(..., (H, 1, 3))`

`seam_report(name, fn, A, B, xs)`

Что делает

вес левого кадра: 1 слева от шва, 0 справа, между ними линейка шириной w ; $w \rightarrow 0$ — срез

строка весов $\rightarrow$ маска $H \times W \times 3$: умножается на цветной кадр, и у неё есть своя пирамида

функция из файла демо: ширина перехода по яркости столбцов и энергия градиента у шва

Демо 2.4 — поиск от грубого к точному

python

```
1 ZNCC = cv2.TM_CCOEFF_NORMED          # tpl — шаблон 15 96 × 96, dark — кадр «другой свет»
2 def search(n):
3     pi, pt = gauss_pyr(dark, n), gauss_pyr(tpl, n)    # пирамиды кадра и шаблона
4     x, y = cv2.minMaxLoc(cv2.matchTemplate(pi[n], pt[n], ZNCC))[3]
5     for k in range(n - 1, -1, -1):                    # вниз: окно ±2 px
6         x, y = max(0, 2 * x - 2), max(0, 2 * y - 2); h, w = pt[k].shape
7         r = cv2.matchTemplate(pi[k][y:y + h + 4, x:x + w + 4], pt[k], ZNCC)
8         dx, dy = cv2.minMaxLoc(r)[3]; x, y = x + dx, y + dy
9     return x, y
10 for n in (0, 2, 3, 5):
11     print("уровней %d: %5.2f мс, ошибка %3.0f px"
12           % (n, ms(lambda: search(n)), err(search(n))))
```

уровней 0: 35 мс, ошибка 0 px · уровней 2:
2.3 мс, 0 px · уровней 3: 0.6–0.8 мс, 0 px
уровней 5: 0.4 мс, ошибка 523 px (времена
плавают)

Вызов

`cv2.minMaxLoc(...)[3]`

`2 * x - 2, окно h + 4, w + 4`

`ms(fn), err(p)`

Что делает

координаты максимума карты отклика — левый верхний угол
найденного окна

координата с уровня выше, удвоенная, и запас ±2 px на
уточнение

время — минимум из пяти прогонов, пирамиды строятся внутри
`search`; ошибка — расстояние до (560, 960)

Фрагмент «оставить низкие частоты»

- `F = np.fft.fft2(img)` — `img` цветной, BGR
- `m[cy-30:cy+30, cx-30:cx+30] = 1`
- `out = np.abs(np.fft.ifft2(F * m))`
- `cv2.imwrite("low.png", out)`

Четыре ошибки

- `fft2` без `axes` на кадре $H \times W \times 3$ берёт оси ($W, 3$): $F[0, 0, 0] = 1.2 \cdot 10^6$ — сумма первой строки, а не канала, $3.6 \cdot 10^8$
- нет `fftshift`: нулевая частота в углу, маска в центре — оставлены самые высокие частоты
- прямоугольная маска — идеальный фильтр: кольца 9 % скачка и значения за пределами 0...255
- `np.abs` и запись без `np.clip`: у фильтра высоких частот модуль перевернёт половину значений

ПОДВОДНЫЕ КАМНИ

- **Забытый сдвиг спектра.** Без `fftshift` маска лежит по центру, а нулевая частота — в углу: фильтр низких частот превращается в фильтр высоких. Проверка — элемент в центре равен сумме пикселей
- **Спектр цветного кадра.** `pr.fft.fft2` берёт две последние оси — ширину и каналы: $1.2 \cdot 10^6$ вместо $3.6 \cdot 10^8$; писать `axes=(0, 1)`
- **Модуль вместо вещественной части.** `pr.abs` после `ifft2` переворачивает отрицательные значения, а у фильтра высоких частот их 50 %; брать `.real` — мнимая часть $4 \cdot 10^{-13}$
- **Резкая маска.** Срез в частотах даёт кольца 9 % скачка при любой ширине и значения от -51 до 291; маска — Гаусс

ПОДВОДНЫЕ КАМНИ

- **Уменьшение без фильтра.** Срез `[::4]` или `INTER_NEAREST` — 21.9 дБ, `INTER_LINEAR` — 27.9 вместо 37.2; `INTER_AREA` или Гаусс $\sigma \approx 0.3$ шага
- **Пирамида Лапласа в байтах.** В `uint8` вычитание `cv2.subtract` обнуляет 49 % значений — кадр восстанавливается на 18.9 дБ; считать во `float32`
- **Увеличение без размера.** `pyrUp` без `dstsize`: $1279 \rightarrow 640 \rightarrow 1280$, формы не сходятся; размер брать у уровня ниже
- **Лишние уровни.** Шаблон 3×3 на вершине — промах на 523 px; в склейке зона $\pm 2^{n+1}$ px шире половины перекрытия — ошибка до 7 кодов

KEY TAKEAWAYS

- Кадр — сумма решёток: спектр хранит амплитуду — сколько — и **фазу — где**; гибрид двух кадров похож на донора фазы, 0.55 против 0.13–0.32; 80 % энергии — в 0.03 % коэффициентов у нулевой частоты
- **Свёртка — умножение спектров** (`filter2D` и `ifft2(F·H)` совпадают до $5 \cdot 10^{-13}$): спектр ядра объясняет фильтр — `box 9 × 9` переворачивает контраст периода 6.3 px (−0.23), `unsharp` усиливает шум в 1.94 раза
- **Прибавленная периодическая помеха — две точки спектра**: `notch` поднимает 22.0 дБ до 34.6 там, где Гаусс даёт 23.8; на растре выигрыш 0.2 дБ; резкая маска звенит на 9 %
- **Выше половины частоты отсчётов — подмена**: период 3 px после среза `[ : : 2 ]` становится 6 px; до уменьшения — фильтр $\sigma \approx 0.3$ шага или `INTER_AREA`: 21.9 → 37.2 дБ
- **Пирамида — спектр, который помнит место**: склейка по полосам даёт переход 134 px при резкости 98 %, поиск по пирамиде быстрее в 60 раз, а лишний уровень — промах на 523 px. В L7 из свёртки вырастут производные и края

САМОПРОВЕРКА

- Кадр повернули на 90° . Что произойдёт с амплитудным спектром — и куда ляжет полоса от вертикалей фасада?
- В кадре решётка с периодом 5 px; кадр уменьшают срезом $[:, :4, :4]$. Какой период получится в пикселях результата и какая σ Гаусса нужна, чтобы подмены не было?
- Ядро $[1, 2, 1] / 4$: чему равна его частотная характеристика на частотах 0, 0.25 и 0.5 цикла на пиксель? Это фильтр низких или высоких частот?
- Два кадра склеивают по полосам, перекрытие 200 px. Сколько уровней пирамиды допустимо и что случится, если взять на два больше?

Разбор — в начале L7, на слайде «Опоры»

ДАЛЬШЕ

- **Занятие 6** (вт 06.10): найти пик помехи в спектре и убрать её notch-фильтром; проверить теорему о свёртке числом; собрать пирамиду Лапласа и склеить два кадра без шва; ★ свой pyrDown без OpenCV
- **Мост:** заготовка начинается на демо 1.4, 1.3, 2.2 и 2.3 этой лекции; рецепт со слайда 47 — в ta-guide
- **ДЗ 1 «Фотолаборатория»** — дедлайн **вс 11.10, 23:59**; **ДЗ 2 «Сканер документов»** выдаётся на этой неделе
- **Quiz 2** (блок II: L4–L6) — на занятии 7, вт 13.10

Ноутбук и данные — в репозитории за сутки до пары

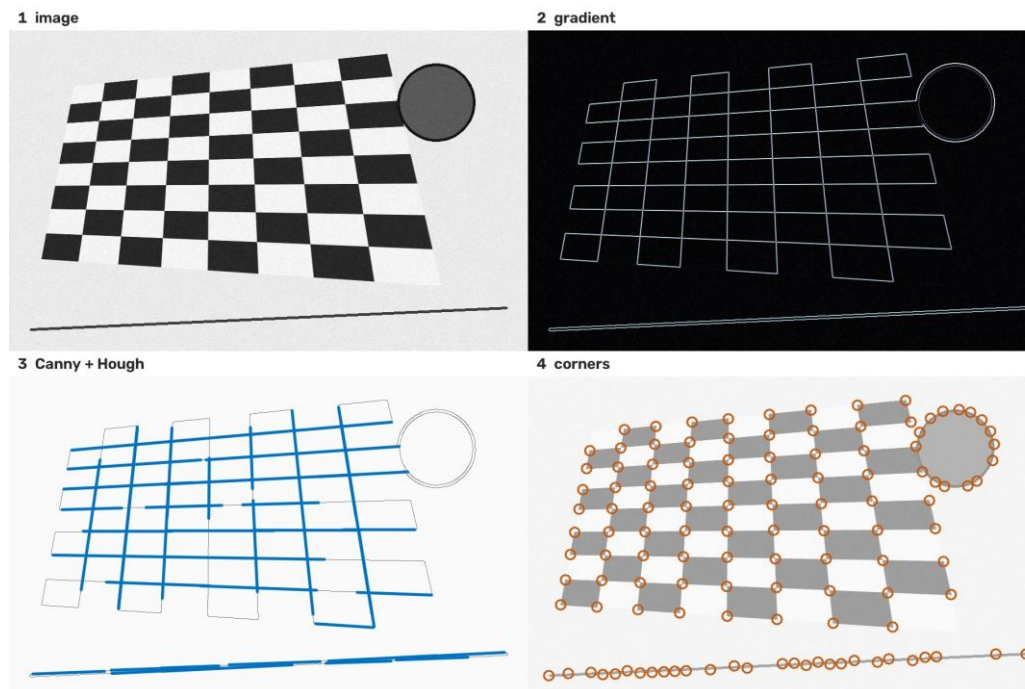
REFERENCES

- Szeliski R. *Computer Vision: Algorithms and Applications*, 2nd ed., 2022 — §3.4 (преобразование Фурье), §3.5 (пирамиды, вейвлеты, склейка); к следующей паре — §7.2 (края) и §7.4 (прямые)
- Гонсалес Р., Вудс Р. *Цифровая обработка изображений*, 3-е изд., 2012 — гл. 4 (фильтрация в частотной области), гл. 7 (вейвлеты и кратномасштабная обработка)
- Burt P., Adelson E. *The Laplacian Pyramid as a Compact Image Code*, IEEE Trans. Communications, 1983 · *A Multiresolution Spline with Application to Image Mosaics*, ACM Trans. Graphics, 1983
- Oppenheim A., Lim J. *The Importance of Phase in Signals*, Proc. IEEE, 1981 · Zhang R. *Making Convolutional Networks Shift-Invariant Again*, ICML 2019
- OpenCV docs 4.x: *Discrete Fourier Transform* (dft, getOptimalDFTSize), *Image Pyramids* (pyrDown, pyrUp), phaseCorrelate; NumPy: `numpy.fft`

ТОЛКОВЫЙ СЛОВАРЬ

Термин	Что это значит
пространственная частота (spatial frequency)	сколько периодов решётки приходится на пиксель или на кадр; мелкие детали — высокие частоты
спектр (spectrum)	коэффициенты разложения кадра по решёткам: амплитуда и фаза для каждой частоты
нулевая частота (DC component)	коэффициент $F(0, 0)$ — сумма всех пикселей, то есть средняя яркость кадра
фаза (phase)	сдвиг решётки относительно начала кадра; хранит, где лежат края
fftshift	перестановка четвертей спектра: нулевая частота из угла переезжает в центр
теорема о свёртке (convolution theorem)	свёртка в пространстве равна умножению спектров кадра и ядра
частотная характеристика (frequency response)	спектр ядра: во сколько раз фильтр меняет амплитуду каждой частоты
режекторный фильтр (notch filter)	провал в маске спектра на частоте помехи и на её зеркальной паре
явление Гиббса (Gibbs ringing)	кольца у краёв после резкого среза частот; перелёт около 9 % скачка
частота Найквиста (Nyquist frequency)	половина частоты отсчётов: 0.5 цикла на пиксель, предел того, что передаёт сетка
алиасинг (aliasing)	подмена частоты выше Найквиста ложной низкой при прореживании без фильтра
пирамида Гаусса (Gaussian pyramid)	цепочка копий кадра, каждая размыта и уменьшена вдвое; вся вместе — 4/3 кадра
пирамида Лапласа (Laplacian pyramid)	разности соседних уровней пирамиды Гаусса: кадр по октавам частот, обратимо
склейка по полосам (multi-band blending)	смешивание уровней пирамид Лапласа по маске своего масштаба: ни шва, ни двоения
вейвлет (wavelet)	короткая волна: базис, локальный и по месту, и по частоте; основа JPEG 2000

Вопросы?



Dr. Ilya Afanasyev · i.afanasyev@innopolis.ru

Демо-код лекции: [lectures/L06-frequency/demo/](#)